

LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

Milestone 4 - Intermediate Code and Interpreter



Oleh

13524014 - Yusuf Faishal Listyardi

13524046 - Farrell Limjaya

13524066 - Nathanael Gunawan

13524070 - A. Fawwaz Azam Wicaksono

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

DAFTAR ISI

BAB I	
PENDAHULUAN.....	1
1.1. Pengantar.....	1
1.2. Spesifikasi.....	2
1.2.1. Spesifikasi Program.....	2
BAB II	
DASAR TEORI.....	24
2.1. Arsitektur Compiler: Frontend dan Backend.....	24
2.2. Decorated AST (Annotated Abstract Syntax Tree).....	25
2.3. Intermediate Representation (IR) & Three Address Code (TAC).....	25
2.4. Set Instruksi Stack Machine.....	26
2.5. Arsitektur Mesin Eksekusi: Interpreter dan Stack Machine.....	29
2.6. Runtime Type Checking.....	30
2.7. Kerentanan Interpreter.....	31
2.8. Compiler vs Interpreter.....	32
BAB III	
PERANCANGAN DAN IMPLEMENTASI.....	34
3.1. Gambaran Umum Program.....	34
3.2. Arsitektur Program.....	35
3.3. Representasi Intermediate Code.....	36
3.4. Implementasi Intermediate Code Generator.....	40
3.5. Translasi Ekspresi.....	49
3.6. Translasi Statement dan Control Flow.....	57
3.7. Implementasi Procedure, Function, dan Stack Frame.....	67
3.8. Implementasi Array, Record, dan Parameter Terstruktur.....	72
3.9. Implementasi Stack Machine Interpreter.....	78
3.10. Implementasi Input dan Output.....	83
3.11. Error Handling dan Diagnostic.....	85
BAB IV	
PENGUJIAN.....	87
4.1. Pengujian 1.....	87
4.2. Pengujian 2.....	88
4.3. Pengujian 3.....	92
4.4. Pengujian 4.....	95
4.5. Pengujian 5.....	97
4.6. Pengujian 6.....	101
4.7. Pengujian 7.....	104
4.8. Pengujian 8.....	107
4.9. Pengujian 9.....	108
4.10. Pengujian 10.....	110
BAB V	
KESIMPULAN DAN SARAN.....	113

5.1. Kesimpulan.....	113
5.2. Saran.....	113
BAB VI	
LAMPIRAN.....	115
6.1. Lampiran Link Repository Github.....	115
6.2. Pembagian Tugas.....	115
REFERENSI.....	116

BAB I

PENDAHULUAN

1.1. Pengantar

Selamat, Anda telah mencapai tahap akhir dari pengembangan bahasa pemrograman Arion!

Mari kita kilas balis sejenak perjalanan source code yang ditulis oleh programmer hingga titik ini sesuai Gambar 1. Pada Milestone I (Lexical Analysis), kode program yang berupa teks mentah telah dipecah menjadi kepingan bermakna yang disebut token. Kemudian pada Milestone II (Syntax Analysis), kumpulan token tersebut disusun ke dalam struktur hierarkis berupa Parse Tree untuk memastikan urutannya sesuai dengan grammar. Selanjutnya pada Milestone III (Semantic Analysis), struktur tersebut divalidasi kebenaran logika dan tipe datanya.

Kini, kita memiliki representasi program yang sudah 100% valid secara sintaksis maupun semantik. Pertanyaannya: Apakah kita sudah bisa menjalankan program tersebut?

Secara teoritis, kita bisa saja membuat program yang langsung menelusuri Parse Tree dari atas ke bawah untuk mengeksekusinya. Namun, dalam dunia nyata, cara ini sangat tidak efisien. Memori komputer dan CPU dirancang untuk mengeksekusi instruksi secara linier atau berurutan (baris demi baris), bukan dengan melompat-lompat di dalam struktur pohon yang kompleks.

Oleh karena itu, pada Milestone IV ini, kita membutuhkan jembatan terakhir sebelum kode benar-benar dieksekusi, yaitu Intermediate Code Generation. Pada tahap ini, Anda akan “meratakan” bentuk pohon yang rumit tadi menjadi daftar instruksi sekuensial yang sangat sederhana dan mirip dengan bahasa mesin, namun masih cukup mudah dibaca oleh manusia. Bentuk perantara inilah yang disebut sebagai Intermediate Code.

Setelah daftar instruksi perantara ini berhasil dibuat, tokoh utama terakhir kita akan mengambil alih: Interpreter. Interpreter bertindak layaknya sebuah virtual machine. Ia akan membaca Intermediate Code tersebut baris demi baris, menyimpan nilai variabel ke dalam memori, mengevaluasi operasi matematika, mengatur ke mana arah jalannya program (jika ada percabangan seperti IF atau perulangan WHILE), dan pada akhirnya mencetak hasil nyata ke layar komputer.

Milestone ini adalah puncak dari Tugas Besar Anda. Semua komponen yang telah Anda bangun secara terpisah pada milestone-milestone sebelumnya akan disatukan. Di akhir tahap ini, bahasa pemrograman Arion tidak lagi sekadar memproses teks, tetapi sudah “hidup” dan mampu memecahkan masalah komputasi layaknya bahasa pemrograman sungguhan seperti Python atau JavaScript.

1.2. Spesifikasi

1.2.1. Spesifikasi Program

Pada milestone ini, Anda diminta untuk membuat dua komponen yaitu Intermediate Code Generator, Stack Penyimpanan, dan Interpreter. Intermediate Code Generator berguna untuk mengubah decorated AST menjadi sebuah langkah-langkah yang dibutuhkan untuk menjalankan program. Informasi-informasi ini akan disimpan dalam stack penyimpanan dan akan digunakan ketika ada operasi tertentu. Lalu berdasarkan kode tersebut, bilamana ada `writeln`, akan keluar hasil akhir dari kode yang dibuat.

Tahapan ini bertujuan untuk mengubah decorated AST menjadi output final dari sebuah kode Arion. Semua tahapan-tahapan sudah membantu anda untuk sampai ke titik ini. Tahapan ini juga bertujuan untuk mendapatkan pemahaman mengenai bagaimana mesin mengelola memori yang berisi informasi-informasi penting seperti variabel, fungsi, prosedur, dll.

Masukan	Decorated AST
Keluaran	Intermediate Code dan output sebenarnya (Jika ada <code>writeln</code>)

Algoritma	Intermediate Code Generation: Depth First Search based approach (Menggunakan prinsip DFS tetapi bukan DFS murni) Interpreter: Stack Machine
-----------	--

Berikut merupakan contoh daftar tipe instruksi, operasi, dan penjelasan dari instruksi tersebut. Ini adalah instruksi-instruksi yang digunakan untuk membuat Intermediate Code. Pada saat mencapai tahap Interpreter, instruksi akan dijalankan pada operasi-operasi yang ada di sebelah kanan instruksi tersebut.

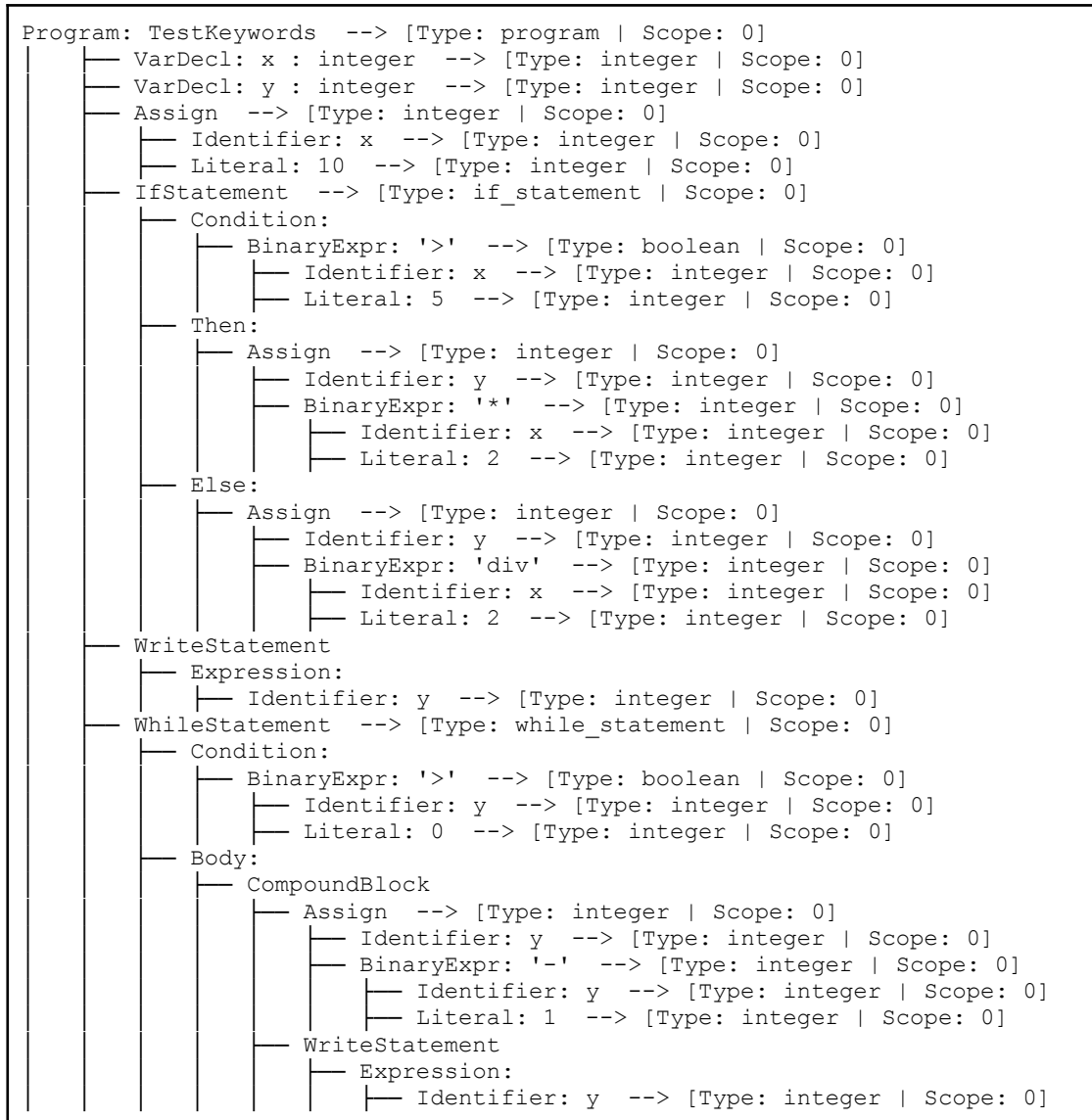
Daftar Instruksi			
No	Instruksi	Operasi	Keterangan
1	LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
2	LOD a	Load Value	Memuat nilai dari address a
3	STO a	Store Value	Menyimpan nilai ke address a
4	CAL l	Call	Memanggil fungsi di garis l
5	INT m	Initiate Memory	Membuat memory dengan ukuran m
6	JMP l	Unconditional Jump	Lompat ke garis l tanpa kondisi apapun
7	JPC l	Conditional Jump	Lompat ke garis l bila kondisi tidak memenuhi
8	OPR o	Operation	Memanggil operasi o (Termasuk fungsi bawaan Arion)
9	RET	Return	Keluar dari fungsi atau prosedur

Berikut juga operasi-operasi yang bisa dijalankan oleh instruksi ‘OPR’

Instruksi OPR (Jalankan seperti ini OPR [value_hierarchy] [operation_no])		
No	Operation Name	Function
1	NEG	Negasi value dari stack teratas
2	ADD	Menambah value stack teratas dengan value stack teratas satunya lagi
3	SUB	Mengurangi value stack teratas dengan value stack teratas satunya lagi
4	MUL	Mengali value stack teratas dengan value stack teratas satunya lagi
5	DIV	Membagi value stack teratas dengan value stack teratas satunya lagi
6	MOD	Modulus value stack teratas dengan value stack teratas satunya lagi
7	EQL	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut sama (Conditionals)
8	NEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut tidak sama (Conditionals)
9	LSS	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari value kedua (Conditionals)
10	GEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari sama dengan value kedua (Conditionals)
11	GTR	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari value kedua (Conditionals)
12	LEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari sama dengan value kedua (Conditionals)
13	WRT	Menulis output yang sudah dimuat
14	WRTLN	Menulis output yang sudah dimuat lalu diberikan newline

Berikut adalah step-by-step cara mengubah decorated AST menjadi sebuah intermediate code:

1. Contoh Decorated AST



2. Definisi Memori

Karena tidak ada prosedur atau fungsi lainnya, akan hanya ada satu inisiasi memori saja. Perhatikan bagian ini

```
Program: TestKeywords --> [Type: program | Scope: 0]
├── VarDecl: x : integer --> [Type: integer | Scope: 0]
└── VarDecl: y : integer --> [Type: integer | Scope: 0]
```


Pada program TestKeywords. Akan dilakukan pendefinisian dua variabel yaitu 'x' dan 'y'. Terdapat bagian dari memori yang sudah akan selalu diisi oleh Static link, Dynamic link, dan Return address. Sisanya bebas digunakan oleh sistem dan biasanya digunakan untuk variabel. Berdasarkan tabel instruksi sebelumnya, sistem intermediate code anda perlu mengembangkan instruksi untuk menginisiasi memori sebanyak 5. Hasilnya adalah seperti ini.

```
0 INT 0 5
```

3. Assign 10 ke x

```
| | | Assign --> [Type: integer | Scope: 0]
| | |   | Identifier: x --> [Type: integer | Scope: 0]
| | |   | Literal: 10 --> [Type: integer | Scope: 0]
```

Melihat instruksi di atas artinya sistem perlu mendefinisikan nilai literal yaitu 10 lalu menyimpannya ke dalam variabel x. Kita tahu bahwa x berada di address 3 sehingga sistem akan mengembangkan perintah untuk memanggil nilai literal 10 lalu menyimpannya di address 3.

```
1 LIT 0 10
2 STO 0 3
```

4. IF statement - Operasi perbandingan

```
| | | IfStatement --> [Type: if_statement | Scope: 0]
| | |   | Condition:
| | |   |   | BinaryExpr: '>' --> [Type: boolean | Scope: 0]
| | |   |   |   | Identifier: x --> [Type: integer | Scope: 0]
| | |   |   |   | Literal: 5 --> [Type: integer | Scope: 0]
```

Instruksi selanjutnya adalah melakukan perbandingan jika x lebih dari 5. x berada di address 3 sehingga sistem mengembangkan perintah untuk memuat value dari address 3. Sistem juga mengembangkan perintah untuk memanggil nilai literal yaitu 5. Setelah itu, untuk melakukan perbandingan, gunakan instruksi OPR ke-11 (GTR) untuk melakukan perbandingan x lebih besar dari 5. Hasilnya akan menjadi seperti ini.

.
.
.
.
.
.
5 - x
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Push nilai literal 5 ke memori

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

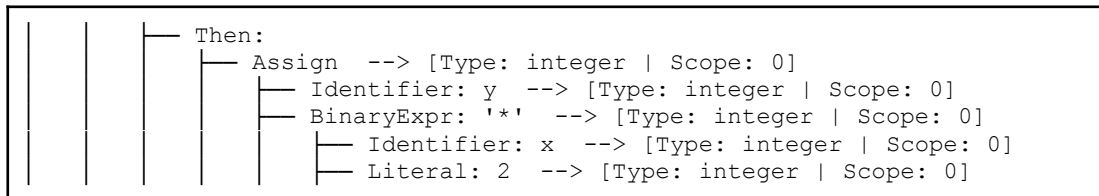
Pop nilai x dan 5 untuk melakukan perbandingan GTR

Instruksi akan terlihat seperti ini

3	LOD	0	3
4	LIT	0	5

```
5 OPR 0 11
```

5. THEN-Statement (Assign $y = x*2$)



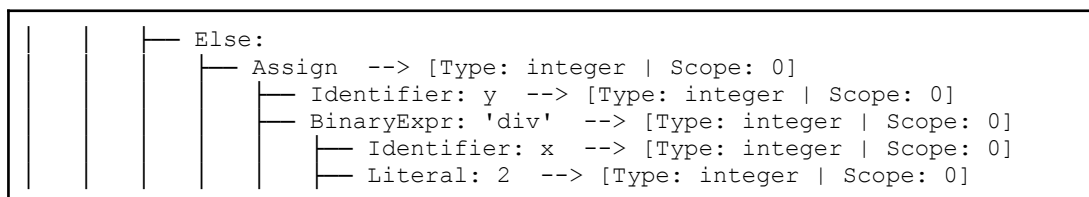
Setelah itu, mari kita lewatkan bagian ‘JPC’ pada line ‘6’ terlebih dahulu dan memeriksa bagian then terlebih dahulu. Tugasnya adalah memberi nilai y sebesar $x*2$. Fokus terlebih dahulu ke $x*2$, melihat instruksi ini. Urutan instruksi untuk sistem yang tepat adalah

- Memuat nilai x (LOD 0 3)
- Memuat nilai literal 2 (LIT 0 2)
- Ambil 2 nilai supaya bisa melakukan perkalian (OPR 0 4)
- Simpan nilai di y (STO 0 4)

Berdasarkan tahapan tersebut, langkah-langkah akan menjadi seperti ini

```
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
```

6. ELSE-Statement (Assign $y = x/2$)



Pada kondisi else. Tugasnya adalah memberi nilai y sebesar $x/2$. Fokus terlebih dahulu ke $x/2$, melihat instruksi ini. Urutan instruksi untuk sistem yang tepat adalah

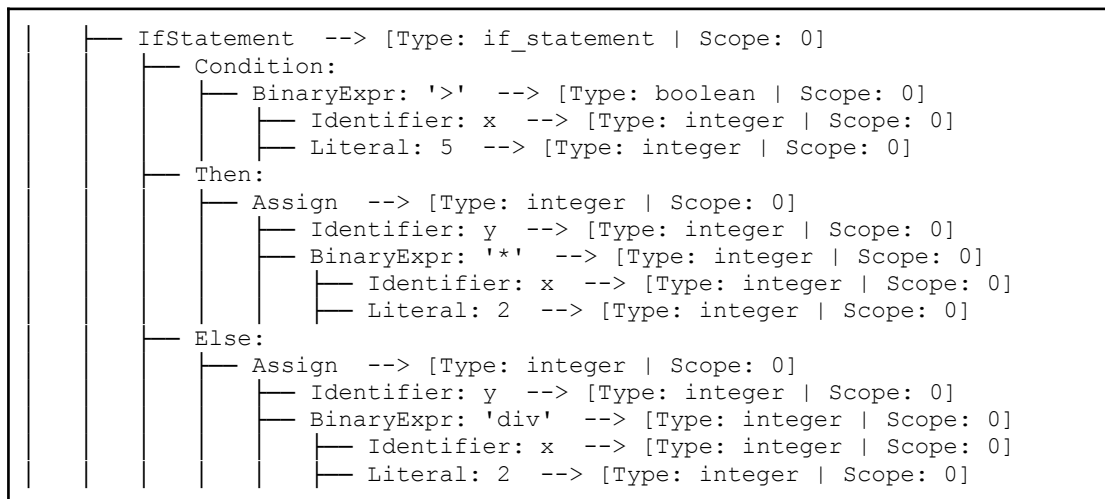
- Memuat nilai x (LOD 0 3)
- Memuat nilai literal 2 (LIT 0 2)

- Ambil 2 nilai supaya bisa melakukan pembagian (OPR 0 5)
- Simpan nilai di y (STO 0 4)

Berdasarkan tahapan tersebut, langkah-langkah akan menjadi seperti ini

```
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4
```

7. Statement lengkap IF-ELSE



Melihat statement-statement yang sudah dibuat, ini artinya 'JPC' memiliki tujuan ke line '12' karena disitulah lokasi 'else' didefinisikan. Setelah line '10' akan didefinisikan juga instruksi untuk lompat ke line '16' yaitu line setelah instruksi ELSE selesai. Oleh karena itu, instruksi blok-IF secara lengkapnya akan terlihat seperti ini

```
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
```

```

14 OPR 0 5
15 STO 0 4

```

8. WRITELN

```

| | | | | WriteStatement
| | | | | | Expression:
| | | | | | | Identifier: y --> [Type: integer | Scope: 0]

```

Bagian ini cukup sederhana. Panggil saja variabel yang akan dijadikan parameter yaitu y lalu panggil OPR ke-14 yaitu Writeln sesuai dengan daftar operasi yang ada di tabel sebelumnya

```

16 LOD 0 4
17 OPR 0 14

```

9. WHILE-Statement

```

| | | | | WhileStatement --> [Type: while_statement | Scope: 0]
| | | | | | Condition:
| | | | | | | BinaryExpr: '>' --> [Type: boolean | Scope: 0]
| | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | Literal: 0 --> [Type: integer | Scope: 0]
| | | | | | Body:
| | | | | | | CompoundBlock
| | | | | | | | Assign --> [Type: integer | Scope: 0]
| | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | | BinaryExpr: '-' --> [Type: integer | Scope: 0]
| | | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | | | Literal: 1 --> [Type: integer | Scope: 0]
| | | | | | | | WriteStatement
| | | | | | | | | Expression:
| | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]

```

Berdasarkan decorated AST ini, kita bisa mengelompokkannya menjadi beberapa instruksi. Berikut adalah instruksi-instruksinya

- Melakukan pemeriksaan $y > 0$ awal. Jika sudah tidak memenuhi syarat langsung hindari While Statement
- Jika memenuhi, muat nilai y dan nilai 1 lalu lakukan operasi pengurangan
- Setelah itu, simpan di address-nya y
- Muat ulang nilai y lalu panggil operasi Writeln
- Lakukan pemeriksaan $y > 0$ dengan lompat kembali ke instruksi pemeriksaan $y > 0$ awal tersebut

Beginilah instruksinya dengan menggunakan Intermediate Code

```
18 LOD 0 4
19 LIT 0 0
20 OPR 0 11
21 JPC 0 29
22 LOD 0 4
23 LIT 0 1
24 OPR 0 3
25 STO 0 4
26 LOD 0 4
27 OPR 0 14
28 JMP 0 18
```

10. Closing

Lakukan return dengan menggunakan RET.

```
29 RET
```

Pada akhirnya, beginilah Intermediate Code secara full.

```
0 INT 0 5
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4
16 LOD 0 4
17 OPR 0 14
18 LOD 0 4
19 LIT 0 0
```

```

20 OPR 0 11
21 JPC 0 29
22 LOD 0 4
23 LIT 0 1
24 OPR 0 3
25 STO 0 4
26 LOD 0 4
27 OPR 0 14
28 JMP 0 18
29 RET

```

Berikut adalah step-by-step dari konversi Intermediate Code ke Interpreter dengan menggunakan Stack Machine

1. Inisiasi Memori

Inisiasi dilakukan dengan mengalokasikan memori sebesar 5. (Sisanya akan digunakan untuk operasi perhitungan, dll)

.
.
.
.
.
.
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 1A: Inisiasi memori awal

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 1B: Inisiasi variabel

2. Simpan nilai 10 ke x

Pada bagian ini, nilai 10 akan ditambahkan ke *stack* machine lalu diambil lagi untuk disimpan ke variabel x di address 3.

.
.
.
.
.
.
5 - Literal 10
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 2A: Menambah nilai literal 10 ke *stack*

.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 2B: Menyimpan nilai literal 10 ke address 3

3. IF-THEN-ELSE statement

.
.
.
.
.
.
6 - literal 5
5 - x
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 3A: Membandingkan nilai x dengan 5. Push nilai x dan 5

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link

0 - Static Link

Tahap 3B: Mengambil nilai x dan 5 untuk dibandingkan

x	>	5
---	---	---

Tahap 3C: Membandingkan jika nilai x lebih besar dari 5

.
.
.
.
.
.
6 - literal 2
5 - x
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4A (Then): Push nilai x dan 2

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)

2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4B (Then): Mengambil nilai x dan 2 supaya bisa dilakukan perkalian

x	*	2
---	---	---

Tahap 4C (Then): x dikali 2

.
.
.
.
.
.
5 - $x*2$
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4D (Then): Memasukkan hasil perkalian ke dalam stack

.
.
.
.
.
.
4 - Variable (y = $x*2$)

3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4E (Then): Memasukkan nilai perkalian ke dalam memori variabel y

.
.
.
.
.
6 - literal 2
5 - x
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5A (Else): Push nilai x dan 2

.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)

2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5B (Else): Mengambil nilai x dan 2 supaya bisa dilakukan pembagian

x	/	2
---	---	---

Tahap 5C (Else): x dibagi 2

.
.
.
.
.
.
5 - $x/2$
4 - Variable (y)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5D (Else): Memasukkan hasil pembagian ke dalam stack

.
.
.
.
.
.
4 - Variable ($y = x/2$)

3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5E (Else): Memasukkan nilai pembagian ke dalam memori variabel y

4. WRITELN

.
.
.
.
.
.
5 - y
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 6A: Memasukkan nilai y ke dalam stack

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 6B: Mengambil nilai y supaya bisa dilakukan operasi yaitu Writeln

20

Tahap 6C: Mengeluarkan output 20

5. WHILE statement

.
.
.
.
.
.
6 - literal 0
5 - y
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 7A: Membandingkan nilai y dengan 0. Push nilai y dan 0

.
.
.
.
.
.

4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 7B: Mengambil nilai y dan 0 untuk dibandingkan

y	>	0
---	---	---

Tahap 7C: Membandingkan jika nilai y lebih besar dari 0

.
.
.
.
.
.
6 - literal 1
5 - y
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8A (Failed to jump): Push nilai y dan 1

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8B (Failed to jump): Mengambil nilai y dan 1 supaya bisa dilakukan pengurangan

y	-	1
---	---	---

Tahap 8C (Failed to jump): y dikurangi 1

.
.
.
.
.
.
5 - y-1
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8D (Failed to jump): Memasukkan hasil pembagian ke dalam stack

.
.
.
.
.
.
4 - Variable ($y = y-1$)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8E (Failed to jump): Memasukkan nilai pembagian ke dalam memori variabel y

Tahap 9: Kembali ke tahap 7 untuk perbandingan. Jika berhasil, ke tahap 10

6. Return

Tahap 10 (Allowed to jump): Return

BAB II

DASAR TEORI

2.1. Arsitektur Compiler: Frontend dan Backend

Sistem compiler modern secara umum dibagi menjadi dua fase arsitektur utama yaitu, frontend dan backend. Frontend mencakup tiga tahapan yang telah diselesaikan pada Milestone I - III

1. Milestone I Lexical Analysis: Teks source code dipecah menjadi satuan bermakna (token)
2. Milestone II Syntax Analysis: Token disusun ke dalam struktur hierarkis Parse Tree untuk memastikan tata bahasa (grammar) terpenuhi
3. Milestone III Semantic Analysis: Struktur pohon divalidasi dari sisi kebenaran logika dan tipe data, menghasilkan Decorated AST

Frontend tidak menghasilkan kode yang bisa dieksekusi, melainkan struktur data murni yang telah terbebas dari teks mentah. Backend adalah yang diimplementasikan pada Milestone IV ini, mencakup dua komponen:

1. Sintesis yang mengonversi Decorated AST menjadi instruksi operasional standar (Intermediate Code).
2. Eksekusi yang mengelola memori dan menjalankan instruksi tersebut pada mesin virtual (Interpreter).

Pemisahan arsitektur yang tegas antara Frontend dan Backend memiliki manfaat penting, jika di kemudian hari sintaks bahasa Arion berubah, cukup Frontend yang dirombak. Selama Frontend baru menghasilkan Decorated AST yang sama, Backend tidak perlu diubah sama sekali.

2.2. Decorated AST (Annotated Abstract Syntax Tree)

Abstract Syntax Tree (AST) adalah representasi pohon hierarkis dari struktur sintaksis kode program. Setelah melewati fase Semantic Analysis, AST tersebut "dihias" atau dianotasi dengan informasi tambahan penting sehingga disebut sebagai Decorated AST. Dekorasi ini memberikan informasi mutlak bagi generator kode perantara, antara lain::

- Informasi Tipe Data (Type Annotations): Menandakan tipe data hasil evaluasi dari setiap node ekspresi (misalnya: integer, real, boolean, char, string). Hal ini krusial agar generator kode dapat memetakan instruksi matematika yang tepat (seperti membedakan pembagian bilangan bulat div dengan pembagian bilangan riil /).
- Referensi Tabel Simbol (Symbol Table References): Setiap variabel memiliki penunjuk (pointer) langsung ke entri tabel simbol untuk mengetahui letak memorinya (address) dan lingkup aksesibilitasnya (scope atau lexical level).

Tanpa adanya Decorated AST, generator kode tidak akan memiliki informasi semantik yang cukup untuk menghasilkan instruksi perantara yang valid.

2.3. Intermediate Representation (IR) & Three Address Code (TAC)

Setelah Frontend selesai memvalidasi program, tugas pertama Backend adalah menerjemahkan representasi konseptual (Decorated AST) ke dalam bentuk instruksi yang siap dieksekusi. Proses inilah yang disebut Intermediate Code Generation.

Alasan mengapa tidak langsung mengeksekusi Parse Tree secara langsung adalah efisiensi, memori komputer dan CPU dirancang untuk mengeksekusi instruksi secara linear (baris demi baris), bukan dengan melompat-lompat di dalam struktur pohon yang kompleks.

Algoritma yang digunakan adalah pendekatan berbasis Depth First Search (DFS) dengan traversal dimulai dari node terdalam (daun) lalu bergerak naik ke akar (post order traversal), menghasilkan instruksi dalam urutan yang benar secara semantik.

Format standar Intermediate Code yang paling luas digunakan adalah Three Address Code (TAC). Dinamakan demikian karena setiap instruksi dirancang sangat sederhana, di mana maksimal hanya melibatkan tiga alamat memori (dua operand sumber dan satu lokasi tujuan).

Struktur dasar TAC: hasil = operand1 OPERATOR operand2

CPU tidak dapat menghitung ekspresi yang sangat panjang sekaligus, sehingga TAC menggunakan variabel sementara untuk memecah ekspresi kompleks menjadi operasi biner sederhana.

Contoh:

$x := (a + b) * (c - d)$

Diratakan menjadi TAC:

$t1 := a + b$ {Langkah 1: Evaluasi kurung pertama}

$t2 := c - d$ {Langkah 2: Evaluasi kurung kedua}

$t3 := t1 * t2$ {Langkah 3: Kalikan hasil keduanya}

$x := t3$ {Langkah 4: Simpan hasil akhir}

Mengubah struktur blok bersarang (nested block) menjadi daftar instruksi linear dilakukan menggunakan kombinasi Label dan Jump (GOTO)

- Label: Penanda statis yang menunjukkan posisi baris tertentu dalam TAC (misal: L1:, L_END:).
- Jump Bersyarat (Conditional Jump / JPC): Melompat ke suatu label hanya jika kondisi yang dievaluasi bernilai False.
- Jump Mutlak (Unconditional Jump / JMP): Melompat langsung ke suatu label tanpa memedulikan kondisi apa pun.

2.4. Set Instruksi Stack Machine

Implementasi Arion Compiler menggunakan model Stack Machine sebagai mesin virtual. Berikut adalah daftar lengkap instruksi yang digunakan:

No	Instruksi	Operasi	Keterangan
1	LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
2	LOD a	Load Value	Memuat nilai dari

			address a ke stack
3	STO a	Store Value	Mengambil nilai dari top stack dan menyimpannya ke address a
4	CAL l	Call	Memanggil fungsi/prosedur yang dimulai di baris l
5	INT m	Initiate Memory	Mengalokasikan memori sebesar m slot untuk stack frame
6	JMP l	Unconditional Jump	Melompat ke baris l tanpa kondisi
7	JPC l	Conditional Jump	Melompat ke baris l bila kondisi di top stack bernilai <i>False</i>
8	OPR o	Operation	Menjalankan operasi nomor o
9	RET	Return	Keluar dari fungsi/prosedur atau mengakhiri program

Operasi-operasi OPR:

No	Nama	Fungsi
1	NEG	Negasi nilai dari top stack
2	ADD	Penjumlahan dua nilai teratas stack

3	SUB	Pengurangan dua nilai teratas stack
4	MUL	Perkalian dua nilai teratas stack
5	DIV	Pembagian dua nilai teratas stack
6	MOD	Modulus dua nilai teratas stack
7	EQL	Perbandingan kesamaan (=)
8	NEQ	Perbandingan ketidaksamaan ($\neq$)
9	LSS	Perbandingan kurang dari ($<$)
10	GEQ	Perbandingan lebih dari atau sama dengan ($\geq$)
11	GTR	Perbandingan lebih dari ($>$)
12	LEQ	Perbandingan kurang dari atau sama dengan ($\leq$)
13	WRT	Menulis output tanpa newline
14	WRTLN	Menulis output dengan newline

2.5. Arsitektur Mesin Eksekusi: Interpreter dan Stack Machine

Setelah Intermediate Code selesai dibuat, program dieksekusi oleh sebuah Interpreter yang bertindak layaknya Virtual Machine (VM). VM tidak memiliki bentuk fisik, tetapi memiliki logika memori dan siklus kerja yang menyerupai CPU nyata.

Sama seperti CPU sungguhan, Interpreter membaca instruksi satu per satu menggunakan penunjuk khusus yang disebut Instruction Pointer (IP) atau Program Counter (PC). IP merupakan variabel yang selalu menunjuk nomor baris instruksi yang sedang dieksekusi.

Interpreter beroperasi dalam sebuah siklus berulang yang terdiri dari tiga tahap:

1. Fetch: Membaca instruksi TAC pada posisi yang ditunjuk oleh IP.
2. Decode: Membedah instruksi untuk mengetahui jenis perintahnya dan siapa operandnya.
3. Execute: Melakukan aksi nyata sesuai instruksi (perhitungan, penyimpanan ke memori, atau lompatan).

Setelah tahap Execute selesai, IP secara default bergerak maju satu langkah ($IP := IP + 1$), kecuali untuk instruksi lompatan (JMP, JPC, CAL, RET) yang mengubah nilai IP secara eksplisit. Siklus ini berulang hingga program selesai.

Untuk menyimpan variabel dan melakukan perhitungan, VM menggunakan struktur data Stack (tumpukan). Stack bekerja dengan prinsip LIFO (Last In, First Out): data yang terakhir dimasukkan (push) adalah data yang pertama diambil (pop).

Setiap kali program memasuki blok kode baru atau memanggil sebuah fungsi/prosedur, Interpreter membuat sebuah ruang memori baru di dalam Stack yang disebut Stack Frame. Layout sebuah Stack Frame adalah sebagai berikut:

Address relatif	Isi
0	Static Link , Pointer ke frame dari enclosing scope (untuk variabel nonlokal)
1	Dynamic Link , Pointer ke base address frame pemanggil (untuk <i>return</i> ke frame sebelumnya)

2	Return Address , Alamat instruksi yang harus dilanjutkan setelah fungsi/prosedur selesai
3+	Variabel Lokal , Diisi oleh variabel yang dideklarasikan di blok tersebut

Instruksi INT m mengalokasikan m slot untuk frame baru. Nilai $m = 3$ (untuk tiga slot krusial di atas) + jumlah variabel lokal.

Mekanisme Stack Frame menjelaskan mengapa variabel lokal di dalam fungsi A tidak dapat diakses oleh fungsi B, VM hanya diperbolehkan membaca dan mengubah variabel yang berada di frame paling atas. Ketika fungsi selesai (RET), framenya dipop dari Stack dan semua variabel lokalnya otomatis musnah.

Untuk mendukung bahasa Arion yang bersifat strongly typed, setiap nilai yang disimpan ke dalam Stack harus memiliki tipe data yang terdefinisi (tagged value). Tipe-tipe yang didukung:

Tipe	Deskripsi
integer	Bilangan bulat
real	Bilangan riil (desimal)
boolean	Nilai logika (true / false)
character	Satu karakter
string	Rangkaian karakter
void	Nilai kosong (untuk prosedur tanpa return value)

2.6. Runtime Type Checking

Meskipun Semantic Analysis (Milestone III) sudah memeriksa tipe data secara statis (sebelum program berjalan), Interpreter perlu berhadapan dengan nilai aktual saat runtime.

Bahasa Arion bersifat *strongly typed*, sehingga runtime type checking tidak diwajibkan. Namun, interpreter tetap harus mampu mendeteksi kondisi-kondisi tidak wajar yang terjadi selama eksekusi, seperti *division by zero* atau operasi antara tipe yang tidak kompatibel, dan menghentikan eksekusi dengan pesan error yang informatif.

2.7. Kerentanan Interpreter

Interpreter yang tangguh tidak hanya tahu cara menjalankan kode yang benar, tetapi juga tahu cara bertahan (tidak crash) ketika diberi kode yang salah atau tidak masuk akal.

Kerentanan	Deskripsi
Stack Overflow	Terlalu banyak Stack Frame <i>dipush</i> hingga melampaui batas memori. Paling umum disebabkan oleh <i>infinite recursion</i> . Interpreter harus membatasi kedalaman Stack (misal: maks. 1000 frame) dan melempar error jika terlampaui.
Stack Underflow	Interpreter diinstruksikan untuk <i>pop</i> dari Stack yang sudah kosong. Biasanya menandakan cacat logika pada Code Generator (misalnya <i>mereturn</i> nilai dari scope global).
Stack Smashing	Bentuk <i>buffer overflow</i> klasik: operasi menulis data melebihi ukuran variabel lokal sehingga menimpa <i>return address</i> di frame yang sama, menyebabkan program melompat ke memori yang salah setelah fungsi selesai.
Stack Corruption	Ketidakseimbangan antara operasi <i>push</i> dan <i>pop</i> : jika satu blok kode melakukan 3 kali <i>push</i> tapi hanya 2 kali <i>pop</i> , "sampah" memori yang tertinggal akan menggeser indeks Stack Frame dan menyebabkan seluruh pembacaan variabel berikutnya menjadi salah.

Karena bahasa Arion mendukung Array, Interpreter wajib memvalidasi setiap akses indeks. Jika array memiliki panjang 5 (indeks 0–4) dan program mencoba mengakses indeks ke-10 atau indeks -1, Interpreter harus mencegah akses tersebut dan melempar `IndexOutOfBoundsException`, bukan meneruskannya ke memori fisik.

Instruksi control flow diubah menjadi JMP dan JPC menuju suatu label. Jika Code Generator membuat kesalahan dan menyuruh Interpreter melompat ke LABEL_X yang tidak ada, Interpreter yang tangguh harus memverifikasi semua target jump sebelum atau saat dieksekusi agar IP tidak "tersesat" ke area memori yang acak.

Tipe data integer memiliki kapasitas maksimum. Jika nilai maksimum int32 dilampaui, akan terjadi wraparound (nilai "tergulung" kembali ke angka negatif besar). Interpreter harus mampu mendeteksi operasi yang akan melampaui batas ini dan menghentikan program dengan OverflowError atau UnderflowError.

2.8. Compiler vs Interpreter

Meskipun keduanya memiliki tujuan yang sama mengeksekusi source code compiler dan interpreter menggunakan mekanisme yang berbeda:

Aspek	Compiler	Interpreter
Waktu Konversi	<i>Ahead-of-Time (AOT)</i> : seluruh source code dikonversi menjadi kode mesin sebelum dieksekusi	<i>On-the-fly</i> : kode dieksekusi langsung saat dibutuhkan, baris per baris
Output	File executable yang dapat dijalankan berulang kali tanpa proses konversi ulang	Tidak menghasilkan file terpisah; eksekusi terjadi dalam satu proses
Kecepatan Eksekusi	Lebih cepat saat runtime karena kode sudah dioptimasi penuh	Lebih lambat karena ada overhead interpretasi tiap instruksi
Portabilitas	Kode mesin bersifat platform-spesifik	Lebih portabel jika menggunakan <i>bytecode</i> sebagai perantara

Beberapa interpreter modern menggunakan bytecode sebagai perantara representasi kode sumber yang bersifat platform-independent dan dieksekusi oleh Virtual Machine (VM) alih-alih mengeksekusi source code secara langsung. Pendekatan ini digunakan oleh bahasa seperti Python, Java, dan Arion.

Selain itu, teknik Just-In-Time (JIT) Compilation menggabungkan keunggulan keduanya: potongan kode dikompilasi ke kode mesin secara on demand (saat pertama kali dieksekusi) dan di cache di memori untuk eksekusi berikutnya. JIT sering lebih cepat dari interpreter bytecode standar karena mengeliminasi overhead interpretasi berulang.

Dalam implementasi Arion Compiler ini, pendekatan yang digunakan adalah interpreter berbasis bytecode (Stack Machine): source code dikompilasi ke Intermediate Code terlebih dahulu oleh Code Generator, kemudian Intermediate Code tersebut dieksekusi baris per baris oleh StackMachineInterpreter.

BAB III

PERANCANGAN DAN IMPLEMENTASI

Implementasi Milestone 4 dibangun di atas hasil semantic analyzer yang telah diimplementasikan pada Milestone 3. Lexer, parser, AST builder, dan semantic analyzer tetap digunakan untuk memproses source code Arion hingga menghasilkan decorated AST dan symbol table yang sudah memuat informasi tipe data, indeks identifier, lexical level, ukuran variabel, parameter, array, dan record. Pada Milestone 4, decorated AST tersebut kemudian digunakan sebagai dasar untuk menghasilkan intermediate code yang bersifat linear dan lebih mudah dieksekusi oleh mesin virtual.

Intermediate code generator bertugas menerjemahkan struktur program pada decorated AST menjadi deretan instruksi stack machine, seperti instruksi alokasi frame, pemuatan literal, load/store variabel, operasi aritmetika dan relasional, percabangan, perulangan, pemanggilan procedure/function, serta instruksi input/output. Setelah intermediate code terbentuk, stack machine interpreter menjalankan instruksi tersebut dengan mengelola memori eksekusi, stack operand, instruction pointer, base frame, static link, dynamic link, return address, parameter, variabel lokal, dan nilai return function. Hasil akhir Milestone 4 berupa semantic diagnostics, intermediate code, code generation diagnostics, output eksekusi program, runtime diagnostics, serta status eksekusi program.

3.1. Gambaran Umum Program

Pada Milestone 4, program dikembangkan dari compiler yang hanya melakukan analisis hingga tahap semantik menjadi compiler sederhana yang mampu menghasilkan intermediate code dan mengeksekusinya. Alur program dimulai dari pembacaan source code Arion, kemudian source code diproses oleh lexical analyzer untuk menghasilkan token. Token tersebut diteruskan ke syntax analyzer untuk membentuk parse tree, lalu parse tree ditransformasikan menjadi Abstract Syntax Tree (AST). AST kemudian dianalisis oleh semantic analyzer untuk memastikan program valid secara semantik dan menghasilkan decorated AST beserta symbol table.

Hasil dari Milestone 3, yaitu decorated AST dan symbol table, menjadi masukan utama bagi tahapan Milestone 4. Decorated AST menyediakan struktur program yang telah diberi informasi semantik, seperti tipe data, indeks symbol table, dan lexical level.

Sementara itu, symbol table menyediakan informasi mengenai identifier, scope, parameter, ukuran variabel, array, dan record. Informasi tersebut digunakan oleh intermediate code generator untuk menerjemahkan program ke dalam deretan instruksi linear berbasis stack machine.

Setelah intermediate code berhasil dibentuk, program menjalankannya menggunakan stack machine interpreter. Interpreter membaca instruksi satu per satu dengan bantuan instruction pointer, mengelola memori eksekusi dalam bentuk stack frame, melakukan operasi aritmetika dan logika, menangani percabangan dan perulangan, menjalankan pemanggilan procedure/function, serta menghasilkan output program.

3.2. Arsitektur Program

Arsitektur program kami dirancang sebagai kelanjutan dari pipeline compiler pada milestone sebelumnya. Komponen frontend yang terdiri dari lexer, parser, AST builder, dan semantic analyzer tetap digunakan sebagai tahap awal untuk memastikan source code valid secara leksikal, sintaksis, dan semantik. Setelah program dinyatakan valid secara semantik, proses dilanjutkan ke komponen backend, yaitu intermediate code generator dan stack machine interpreter.

Komponen intermediate code generator diimplementasikan melalui kelas CodeGenerator. Kelas ini menerima root AST dan symbol table hasil semantic analysis, lalu menghasilkan objek IntermediateProgram. Di dalam prosesnya, CodeGenerator memecah pembangkitan kode berdasarkan jenis node AST, seperti deklarasi, statement, dan ekspresi. Implementasi generator dipisahkan ke beberapa file, antara lain codegen.cpp, codegen_declaration.cpp, codegen_statement.cpp, dan codegen_expression.cpp. Pemisahan ini membuat tanggung jawab setiap bagian lebih jelas dan memudahkan pengembangan fitur seperti control flow, routine call, array, record, serta input/output.

Representasi intermediate code didefinisikan melalui struktur Instruction, OpCode, OperationCode, dan IntermediateProgram. Setiap instruksi menyimpan opcode, level leksikal, operand, literal jika diperlukan, serta penanda indirect access. Instruksi-instruksi tersebut merepresentasikan operasi sederhana yang dapat dijalankan oleh stack machine, seperti alokasi memori, load/store nilai, operasi aritmetika, percabangan, pemanggilan routine, dan return. Dengan representasi ini, struktur AST yang berbentuk pohon dapat diterjemahkan menjadi instruksi linear yang lebih mudah dieksekusi.

Komponen eksekusi diimplementasikan melalui kelas `StackMachineInterpreter`. Interpreter menerima `IntermediateProgram` dari code generator, kemudian menjalankan setiap instruksi menggunakan model mesin stack. Selama eksekusi, interpreter mengelola memory sebagai tempat penyimpanan frame dan variabel, stack sebagai operand stack, ip sebagai instruction pointer, serta base sebagai penunjuk frame aktif. Interpreter juga menggunakan metadata routine untuk menangani pemanggilan procedure dan function, termasuk static link, dynamic link, return address, parameter, variabel lokal, dan nilai return function.

Integrasi seluruh komponen dilakukan melalui modul driver dan CLI. Driver menjalankan tahapan compiler secara berurutan, mulai dari lexical analysis hingga eksekusi intermediate code. Jika ditemukan error pada tahap semantic analysis, code generator dan interpreter tidak dijalankan. Jika code generation gagal, interpreter juga tidak dijalankan. Dengan alur ini, program dapat menghasilkan output yang terstruktur berupa semantic diagnostics, intermediate code, codegen diagnostics, program output, runtime diagnostics, dan status akhir eksekusi.

3.3. Representasi Intermediate Code

Intermediate code pada program kami direpresentasikan sebagai deretan instruksi linear yang dijalankan oleh stack machine interpreter. Representasi ini digunakan untuk meratakan struktur decorated AST yang semula berbentuk pohon menjadi bentuk instruksi berurutan. Setiap node penting pada AST, seperti ekspresi, assignment, control flow, procedure call, dan function call, diterjemahkan menjadi satu atau lebih instruksi intermediate. Dengan demikian, interpreter tidak perlu lagi memahami struktur AST, melainkan cukup menjalankan instruksi satu per satu berdasarkan instruction pointer.

Setiap instruksi intermediate direpresentasikan oleh struktur `Instruction`. Struktur ini menyimpan beberapa informasi utama, yaitu opcode, level leksikal, operand, literal, penanda keberadaan literal, penanda indirect access, serta komentar opsional. Opcode menentukan jenis operasi yang dilakukan, misalnya load, store, jump, call, atau operasi aritmetika. Level leksikal digunakan untuk mengakses variabel pada scope yang berbeda melalui static link. Operand digunakan sebagai alamat, target jump, ukuran alokasi, kode operasi, atau informasi tambahan lain sesuai opcode. Literal digunakan khusus untuk instruksi yang memasukkan nilai konstan ke stack. Jenis instruksi utama direpresentasikan oleh enumerasi `OpCode`. Instruksi `INT` digunakan untuk mengalokasikan memori

frame, sedangkan `LIT` digunakan untuk memasukkan nilai literal ke operand stack. Instruksi `LOD` dan `STO` digunakan untuk mengambil dan menyimpan nilai scalar dari atau ke memori. Untuk kebutuhan akses tidak langsung, array, dan record, digunakan instruksi `LDA`, `LDI`, dan `STI`, yaitu untuk mengambil alamat, mengambil nilai berdasarkan alamat, dan menyimpan nilai ke alamat tertentu. Selain itu, terdapat instruksi `CPY` untuk menyalin beberapa slot memori pada assignment array atau record, serta `LDR` untuk memuat nilai aggregate sebagai argumen by-value pada pemanggilan routine.

Control flow direpresentasikan dengan instruksi `JMP` dan `JPC`. Instruksi `JMP` melakukan lompatan tanpa syarat ke alamat instruksi tertentu, sedangkan `JPC` melakukan lompatan jika nilai kondisi yang diambil dari stack bernilai false. Dengan dua instruksi ini, struktur tingkat tinggi seperti `if-else`, `while`, `repeat-until`, `for`, dan `case` dapat diterjemahkan menjadi alur instruksi linear. Pemanggilan procedure dan function direpresentasikan dengan instruksi `CAL`, sedangkan akhir eksekusi routine direpresentasikan dengan `RET`.

Operasi aritmetika, relasional, Boolean, dan output direpresentasikan melalui instruksi `OPR` dengan kode operasi yang didefinisikan dalam `OperationCode`. Beberapa operasi yang didukung antara lain `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `RDIV`, `EQL`, `NEQ`, `LSS`, `LEQ`, `GTR`, `GEQ`, `NEG`, `WRT`, dan `WRTLN`. Dengan desain ini, satu opcode `OPR` dapat digunakan untuk berbagai operasi yang berbeda, sementara detail operasinya ditentukan oleh operand instruksi.

Selain itu, kami juga menambahkan instruksi `INP` dan `INL` untuk mendukung prosedur bawaan `read` dan `readln`. Instruksi `INP` membaca satu token input dari pengguna, mengubahnya sesuai tipe target, lalu menyimpannya ke alamat memori yang diberikan. Instruksi `INL` digunakan untuk membuang sisa input pada baris berjalan atau membaca satu baris kosong, sehingga perilaku `readln` dapat dibedakan dari `read`. Dengan representasi intermediate code ini, program dapat memisahkan proses translasi source code dari proses eksekusinya secara lebih modular.

Representasi Intermediate Code Secara Langsung

```
enum class RuntimeValueKind {
```



```

    Void,
    Integer,
    Real,
    Boolean,
    Char,
    String
};

struct RuntimeValue {
    RuntimeValueKind kind = RuntimeValueKind::Void;
    int integerValue = 0;
    double realValue = 0.0;
    bool booleanValue = false;
    char charValue = '\\0';
    std::string stringValue;

    static RuntimeValue voidValue();
    static RuntimeValue integer(int value);
    static RuntimeValue real(double value);
    static RuntimeValue boolean(bool value);
    static RuntimeValue character(char value);
    static RuntimeValue string(std::string value);

    bool isTruthy() const;
    std::string display() const;
};

enum class OpCode {
    LIT,
    LOD,
    LDA,
    LDI,
    LDR,
    STO,
    STI,
    CPY,
    INP,
    INL,
    CAL,
    INT,
    JMP,
    JPC,
    OPR,
    RET
};

```

```

enum class OperationCode {
    NEG = 1,
    ADD = 2,
    SUB = 3,
    MUL = 4,
    DIV = 5,
    MOD = 6,
    EQL = 7,
    NEQ = 8,
    LSS = 9,
    GEQ = 10,
    GTR = 11,
    LEQ = 12,
    WRT = 13,
    WRTLN = 14,
    RDIV = 15
};

struct Instruction {
    OpCode opcode = OpCode::RET;
    int level = 0;
    int operand = 0;
    RuntimeValue literal;
    bool hasLiteral = false;
    bool indirect = false;
    std::string comment;
};

struct RoutineMetadata {
    int address = 0;
    int parameterCount = 0;
    int frameSize = 0;
    bool returnsValue = false;
    int returnValueOffset = 0;
    std::vector<bool> byReferenceParameters;
};

struct IntermediateProgram {
    std::vector<Instruction> instructions;
    std::vector<RoutineMetadata> routines;
};

std::string opcodeName(OpCode opcode);
std::string operationName(OperationCode operation);
std::string formatInstruction(const Instruction& instruction, int address);
std::vector<std::string> formatProgram(const IntermediateProgram& program);

```

3.4. Implementasi Intermediate Code Generator

Intermediate Code Generator diimplementasikan melalui kelas CodeGenerator. Generator tidak lagi melakukan pengecekan semantik dari awal, melainkan memanfaatkan informasi yang sudah tersedia pada decorated AST dan symbol table, seperti tipe data, indeks identifier pada tab, lexical level, ukuran variabel, ukuran parameter, serta informasi array dan record pada atab dan btab. Dengan demikian, proses pembangkitan kode dapat dilakukan berdasarkan informasi semantik yang sudah valid.

Proses pembangkitan intermediate code dimulai dari pemanggilan method generate(). Method ini menerima root AST dan symbol table, kemudian menginisialisasi state internal code generator sebelum memproses program utama. Program diproses melalui generateProgram(), yang bertanggung jawab mengatur alokasi memori awal, memproses deklarasi procedure dan function, membangkitkan instruksi untuk body program, serta menambahkan instruksi RET pada akhir program. Jika program memiliki deklarasi routine, generator terlebih dahulu membangkitkan instruksi JMP agar eksekusi utama tidak langsung masuk ke body procedure atau function sebelum dipanggil.

Proses code generation dipisahkan berdasarkan jenis konstruksi program. Deklarasi diproses pada bagian codegen_declaration, statement diproses pada codegen_statement, sedangkan ekspresi diproses pada codegen_expression. Deklarasi variabel, konstanta, dan tipe tidak selalu menghasilkan instruksi runtime karena informasinya sudah direpresentasikan pada symbol table. Sebaliknya, deklarasi procedure dan function menghasilkan alamat entry routine, metadata frame, instruksi alokasi frame, body routine, dan instruksi RET.

Pada pembangkitan statement, generator menerjemahkan struktur tingkat tinggi menjadi instruksi jump dan operasi stack machine. Assignment scalar diterjemahkan menjadi evaluasi ekspresi diikuti instruksi store. Jika target assignment berupa array atau record, generator menggunakan mekanisme copy aggregate dengan instruksi CPY. Statement if, while, repeat, for, dan case diterjemahkan menggunakan kombinasi JMP dan JPC. Instruksi jump yang targetnya belum diketahui pada saat awal pembangkitan akan disimpan terlebih dahulu, kemudian diperbaiki menggunakan mekanisme patching setelah alamat tujuan diketahui.

Pada pembangkitan ekspresi, generator menggunakan pendekatan stack-based evaluation. Literal diterjemahkan menjadi instruksi LIT, variabel scalar menjadi LOD, sedangkan

operator aritmetika, relasional, dan Boolean diterjemahkan menjadi instruksi OPR dengan kode operasi tertentu. Untuk akses array dan record, generator terlebih dahulu menghitung alamat efektif menggunakan LDA dan operasi penjumlahan offset, lalu menggunakan LDI atau STI untuk membaca atau menulis nilai melalui alamat tersebut. Pendekatan ini memungkinkan elemen array, field record, serta kombinasi array of record diproses secara seragam melalui mekanisme alamat.

Code generator juga menangani pemanggilan procedure dan function melalui instruksi CAL. Sebelum CAL dibangkitkan, seluruh argumen dievaluasi dan diletakkan pada stack. Untuk parameter by-value scalar, nilai ekspresi langsung dimasukkan ke stack. Untuk parameter by-reference, generator membangkitkan alamat argumen agar routine dapat mengakses variabel asli pemanggil. Untuk parameter by-value yang bertipe array atau record, generator menggunakan instruksi LDR untuk menyalin seluruh slot aggregate ke stack, sehingga perubahan terhadap parameter di dalam routine tidak memengaruhi variabel asli pemanggil.

Selain itu, code generator juga mendukung procedure bawaan write, writeln, read, dan readln. Pemanggilan write dan writeln diterjemahkan menjadi instruksi OPR dengan operasi output, sedangkan read dan readln diterjemahkan menjadi instruksi input INP dan INL. Dengan pemisahan ini, operasi input/output tetap direpresentasikan sebagai bagian dari intermediate code dan dapat dijalankan oleh interpreter secara konsisten.

Hasil akhir dari proses code generation adalah objek IntermediateProgram yang berisi daftar instruksi dan metadata routine. Daftar instruksi digunakan oleh interpreter untuk menjalankan program secara linear, sedangkan metadata routine digunakan untuk membentuk stack frame ketika terjadi pemanggilan procedure atau function. Jika terjadi kesalahan saat pembangkitan kode, generator mencatat pesan pada codegen diagnostics dan interpreter tidak dijalankan. Dengan rancangan ini, intermediate code generator menjadi penghubung antara decorated AST dari frontend dan stack machine interpreter pada backend.

Entry point code generator dan alur program utama

```
CodeGenerationResult CodeGenerator::generate(const ProgramNode& root, const
SymbolTable& symbols) {
```

```

    reset(symbols);

    try {
        generateProgram(root);
    } catch (const std::exception& error) {
        diagnostic("internal code generation error: " +
std::string(error.what()), root.location);
    } catch (...) {
        diagnostic("internal code generation error: unknown exception",
root.location);
    }

    return result;
}

void CodeGenerator::reset(const SymbolTable& symbols) {
    symbolTable = &symbols;
    result = {};
    routineAddressesByTabIndex.assign(symbols.tab().size(), -1);
    currentBlockIndex = 0;
    currentLexLevel = 0;
    currentFunctionTabIndex = 0;
    currentFunctionReturnOffset = 0;
}

void CodeGenerator::generateProgram(const ProgramNode& node) {
    if (!symbolTable) {
        diagnostic("missing symbol table for code generation", node.location);
        return;
    }

    int variableCount = 0;
    if (node.blockIndex >= 0 && node.blockIndex <
static_cast<int>(symbolTable->btabs().size())) {
        variableCount = symbolTable->btabsAt(node.blockIndex).vsizes;
    } else {
        diagnostic("program block index is out of range", node.location);
    }

    if (variableCount < 0) {
        diagnostic("program variable size is negative", node.location);
        variableCount = 0;
    }

    int mainJump = -1;
    bool hasRoutineDeclarations = false;

```

```

for (const auto& declaration : node.declarations) {
    if (!declaration) {
        continue;
    }

    if (declaration->kind == ASTNodeKind::ProcDecl ||
        declaration->kind == ASTNodeKind::FuncDecl) {
        hasRoutineDeclarations = true;
        break;
    }
}

if (hasRoutineDeclarations) {
    Instruction jump;
    jump.opcode = OpCode::JMP;
    mainJump = emit(jump);
}

for (const auto& declaration : node.declarations) {
    if (!declaration) {
        continue;
    }

    generateDeclaration(*declaration);
}

if (mainJump >= 0) {
    patchOperand(mainJump, nextAddress());
}

currentBlockIndex = node.blockIndex;
currentLexLevel = 0;
currentFunctionTabIndex = 0;
currentFunctionReturnOffset = 0;

Instruction init;
init.opcode = OpCode::INT;
init.operand = frameHeaderSize + variableCount;
emit(init);

if (node.body) {
    generateBlock(*node.body);
} else {
    diagnostic("program body is missing", node.location);
}

```

```

Instruction ret;
ret.opcode = OpCode::RET;
emit(ret);
}

```

Dispatcher deklarasi dan pembangkitan procedure/function

```

void CodeGenerator::generateDeclaration(const DeclarationNode& node) {
    switch (node.kind) {
        case ASTNodeKind::VarDecl:
            generateVarDecl(static_cast<const VarDeclNode&>(node));
            break;
        case ASTNodeKind::ConstDecl:
            generateConstDecl(static_cast<const ConstDeclNode&>(node));
            break;
        case ASTNodeKind::TypeDecl:
            generateTypeDecl(static_cast<const TypeDeclNode&>(node));
            break;
        case ASTNodeKind::ProcDecl:
            generateProcDecl(static_cast<const ProcDeclNode&>(node));
            break;
        case ASTNodeKind::FuncDecl:
            generateFuncDecl(static_cast<const FuncDeclNode&>(node));
            break;
        default:
            diagnostic("unsupported declaration node for code generation",
node.location);
            break;
    }
}

void CodeGenerator::generateVarDecl(const VarDeclNode& /*node*/) {
    // Variable declarations are materialized in btab/tab by semantic
analysis.
    // Program-level allocation is emitted once by generateProgram via INT.
}

void CodeGenerator::generateConstDecl(const ConstDeclNode& /*node*/) {
    // Constants do not emit runtime instructions; expressions load them as
LIT.
}

void CodeGenerator::generateTypeDecl(const TypeDeclNode& /*node*/) {
    // Types only guide semantic annotation and address calculations.
}

```

```

}

void CodeGenerator::generateProcDecl(const ProcDeclNode& node) {
    if (!symbolTable) {
        diagnostic("missing symbol table for procedure declaration",
node.location);
        return;
    }

    if (node.blockIndex <= 0 || node.blockIndex >=
static_cast<int>(symbolTable->btabs().size())) {
        diagnostic("procedure block index is out of range for '" + node.name +
"', node.location);
        return;
    }

    const BTabEntry& block = symbolTable->btabsAt(node.blockIndex);
    const int address = nextAddress();
    registerRoutine(node.tabIndex, address, block, false);

    const int savedBlock = currentBlockIndex;
    const int savedLevel = currentLexLevel;
    const int savedFunctionTab = currentFunctionTabIndex;
    const int savedFunctionReturnOffset = currentFunctionReturnOffset;

    currentBlockIndex = node.blockIndex;
    currentLexLevel = node.lexLevel + 1;
    currentFunctionTabIndex = 0;
    currentFunctionReturnOffset = 0;

    Instruction init;
    init.opcode = OpCode::INT;
    init.operand = frameHeaderSize + block.psize + block.vsize;
    emit(init);

    int bodyJump = -1;
    if (hasRoutineDeclaration(node.declarations)) {
        Instruction jump;
        jump.opcode = OpCode::JMP;
        bodyJump = emit(jump);
    }

    for (const auto& declaration : node.declarations) {
        if (declaration) {
            generateDeclaration(*declaration);
        }
    }
}

```



```

    }

    if (bodyJump >= 0) {
        patchOperand(bodyJump, nextAddress());
    }

    if (node.body) {
        generateBlock(*node.body);
    } else {
        diagnostic("procedure '" + node.name + "' is missing its body",
node.location);
    }

    Instruction ret;
    ret.opcode = OpCode::RET;
    emit(ret);

    currentBlockIndex = savedBlock;
    currentLexLevel = savedLevel;
    currentFunctionTabIndex = savedFunctionTab;
    currentFunctionReturnOffset = savedFunctionReturnOffset;
}

```

Dispatcher statement dan assignment aggregate

```

void CodeGenerator::generateStatement(const StatementNode& node) {
    switch (node.kind) {
        case ASTNodeKind::Block:
            generateBlock(static_cast<const BlockNode&>(node));
            break;
        case ASTNodeKind::Assign:
            generateAssignment(static_cast<const AssignNode&>(node));
            break;
        case ASTNodeKind::If:
            generateIf(static_cast<const IfNode&>(node));
            break;
        case ASTNodeKind::While:
            generateWhile(static_cast<const WhileNode&>(node));
            break;
        case ASTNodeKind::Repeat:
            generateRepeat(static_cast<const RepeatNode&>(node));
            break;
        case ASTNodeKind::For:
            generateFor(static_cast<const ForNode&>(node));
    }
}

```

```

        break;
    case ASTNodeKind::Case:
        generateCase(static_cast<const CaseNode&>(node));
        break;
    case ASTNodeKind::ProcCall:
        generateProcedureCall(static_cast<const ProcCallNode&>(node));
        break;
    default:
        diagnostic("unsupported statement node for code generation",
node.location);
        break;
    }
}

void CodeGenerator::generateAssignment(const AssignNode& node) {
    if (!node.target || !node.value) {
        diagnostic("assignment is missing its target or value",
node.location);
        return;
    }

    const TypeInfo targetType = expressionTypeInfo(*node.target);
    if (isStructuredType(targetType)) {
        if (!symbolTable) {
            diagnostic("missing symbol table for structured assignment",
node.location);
            return;
        }

        const TypeInfo valueType = expressionTypeInfo(*node.value);
        if (!isStructuredType(valueType)) {
            diagnostic("structured assignment source is not an array or
record", node.value->location);
            return;
        }

        const int copySize = symbolTable->sizeof(targetType);
        if (copySize <= 0) {
            diagnostic("structured assignment has invalid copy size",
node.location);
            return;
        }

        size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(*node.value);
        if (result.diagnostics.size() != diagnosticCount) {

```

```

        return;
    }

    diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*node.target);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    Instruction instruction;
    instruction.opcode = OpCode::CPY;
    instruction.operand = copySize;
    emit(instruction);
    return;
}

```

Control flow dengan patching jump

```

void CodeGenerator::generateIf(const IfNode& node) {
    if (!node.condition || !node.thenBranch) {
        diagnostic("if statement is missing its condition or then-branch",
node.location);
        return;
    }

    const size_t diagnosticCount = result.diagnostics.size();
    generateExpression(*node.condition);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    const int falseJump = emit(makeJump(OpCode::JPC));
    generateStatement(*node.thenBranch);

    if (node.elseBranch) {
        const int endJump = emit(makeJump(OpCode::JMP));
        patchOperand(falseJump, nextAddress());
        generateStatement(*node.elseBranch);
        patchOperand(endJump, nextAddress());
    } else {
        patchOperand(falseJump, nextAddress());
    }
}

```

```

void CodeGenerator::generateWhile(const WhileNode& node) {
    if (!node.condition || !node.body) {
        diagnostic("while statement is missing its condition or body",
node.location);
        return;
    }

    const int loopStart = nextAddress();
    const size_t diagnosticCount = result.diagnostics.size();
    generateExpression(*node.condition);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    const int exitJump = emit(makeJump(OpCode::JPC));
    generateBlock(*node.body);
    emit(makeJump(OpCode::JMP, loopStart));
    patchOperand(exitJump, nextAddress());
}

```

3.5. Translasi Ekspresi

Translasi ekspresi pada program kami dilakukan oleh intermediate code generator dengan pendekatan stack-based evaluation. Setiap ekspresi pada decorated AST diterjemahkan menjadi deretan instruksi yang akan menempatkan hasil evaluasi ekspresi pada operand stack. Dengan pendekatan ini, operasi yang bersifat rekursif pada AST dapat diratakan menjadi instruksi linear. Subekspresi akan dibangkitkan terlebih dahulu, kemudian hasilnya digunakan oleh instruksi operasi yang sesuai.

Untuk ekspresi literal, generator langsung membangkitkan instruksi `LIT`. Literal integer, real, char, string, dan boolean dikonversi menjadi `RuntimeValue` sesuai jenisnya, lalu dimasukkan ke stack melalui instruksi tersebut. Sebagai contoh, literal integer akan diterjemahkan menjadi `LIT` dengan nilai integer, sedangkan literal string akan diterjemahkan menjadi `LIT` dengan nilai string. Dengan cara ini, interpreter cukup membaca instruksi `LIT` untuk memasukkan nilai konstan ke operand stack.

Untuk ekspresi variabel, generator membangkitkan instruksi load berdasarkan jenis aksesnya. Variabel scalar biasa diterjemahkan menjadi instruksi `LOD`, sedangkan variabel yang merupakan parameter by-reference dibaca secara indirect menggunakan

alamat yang tersimpan pada frame. Jika ekspresi berupa akses array atau record, generator terlebih dahulu menghitung alamat efektif dari elemen atau field yang diakses. Setelah alamat tersebut diperoleh, nilai diambil menggunakan instruksi `'LDI'`. Perhitungan alamat efektif ini memanfaatkan informasi dari symbol table, seperti alamat dasar variabel, batas bawah array, ukuran elemen array, serta offset field record.

Ekspresi unary diterjemahkan dengan membangkitkan ekspresi operand terlebih dahulu, kemudian membangkitkan instruksi operasi yang sesuai. Operator unary `'+'` tidak menghasilkan operasi tambahan karena nilai operand dapat langsung digunakan. Operator unary `'-'` diterjemahkan menjadi instruksi `'OPR NEG'`, sedangkan operator `'not'` diterjemahkan dengan membandingkan operand terhadap nilai boolean `'false'`. Dengan demikian, hasil akhir ekspresi unary tetap berada di operand stack.

Ekspresi binary diterjemahkan dengan membangkitkan kode untuk operand kiri dan operand kanan secara berurutan, kemudian membangkitkan instruksi `'OPR'` sesuai operator yang digunakan. Operator aritmetika seperti `'+'`, `'-'`, `'*'`, `'div'`, `'mod'`, dan `'/'` dipetakan ke operasi `'ADD'`, `'SUB'`, `'MUL'`, `'DIV'`, `'MOD'`, dan `'RDIV'`. Operator relasional seperti `'=='`, `'<>'`, `'<'`, `'<='`, `'>'`, dan `'>='` dipetakan ke operasi `'EQL'`, `'NEQ'`, `'LSS'`, `'LEQ'`, `'GTR'`, dan `'GEQ'`. Operasi tersebut kemudian dieksekusi oleh interpreter dengan mengambil operand dari stack dan memasukkan hasil operasi kembali ke stack.

Untuk operator Boolean `'and'` dan `'or'`, generator menerapkan short-circuit evaluation. Pada operator `'and'`, operand kanan hanya dievaluasi apabila operand kiri bernilai true. Jika operand kiri bernilai false, hasil akhir langsung false tanpa mengevaluasi operand kanan. Sebaliknya, pada operator `'or'`, hasil akhir langsung true apabila operand kiri bernilai true, sehingga operand kanan tidak perlu dievaluasi. Translasi ini dilakukan menggunakan kombinasi instruksi `'JPC'`, `'JMP'`, literal boolean, dan patching alamat jump. Pendekatan ini penting karena mencegah evaluasi ekspresi yang tidak diperlukan, misalnya ekspresi yang dapat menyebabkan division by zero pada operand kanan.

Pemanggilan function sebagai ekspresi juga ditangani pada tahap translasi ekspresi. Generator terlebih dahulu mencari metadata function berdasarkan symbol table, kemudian membangkitkan kode untuk seluruh argumen. Untuk argumen by-value, nilai argumen dimasukkan ke stack. Untuk argumen by-reference, alamat argumen dimasukkan ke stack. Jika argumen bertipe array atau record dan dikirim by-value, generator menggunakan instruksi `'LDR'` untuk memuat seluruh slot aggregate ke stack. Setelah seluruh argumen

siap, generator membangkitkan instruksi 'CAL' untuk memanggil function. Setelah function selesai dieksekusi, nilai return function diletakkan kembali di operand stack sehingga dapat digunakan sebagai bagian dari ekspresi lain.

Dengan rancangan ini, translasi ekspresi dapat menangani berbagai bentuk ekspresi Arion secara konsisten, mulai dari literal sederhana, variabel, operasi aritmetika dan logika, akses array dan record, hingga pemanggilan function. Seluruh hasil ekspresi selalu mengikuti prinsip yang sama, yaitu nilai akhir berada di operand stack dan siap digunakan oleh instruksi berikutnya, seperti assignment, output, kondisi control flow, atau operasi ekspresi yang lebih besar.

Dispatcher ekspresi utama
<pre> void CodeGenerator::generateExpression(const ExpressionNode& node) { switch (node.kind) { case ASTNodeKind::IntLiteral: { const auto& literal = static_cast<const IntLiteralNode&>(node); emit(makeLiteral(RuntimeValue::integer(literal.value))); break; } case ASTNodeKind::RealLiteral: { const auto& literal = static_cast<const RealLiteralNode&>(node); emit(makeLiteral(RuntimeValue::real(literal.value))); break; } case ASTNodeKind::CharLiteral: { const auto& literal = static_cast<const CharLiteralNode&>(node); emit(makeLiteral(RuntimeValue::character(literal.value))); break; } case ASTNodeKind::StringLiteral: { const auto& literal = static_cast<const StringLiteralNode&>(node); emit(makeLiteral(RuntimeValue::string(literal.value))); break; } case ASTNodeKind::BoolLiteral: { const auto& literal = static_cast<const BoolLiteralNode&>(node); emit(makeLiteral(RuntimeValue::boolean(literal.value))); break; } case ASTNodeKind::Var: case ASTNodeKind::ArrayAccess: case ASTNodeKind::RecordAccess: </pre>

```
emitLoadAddressable(node);
break;
```

Unary, binary, dan short-circuit Boolean

```
case ASTNodeKind::UnaryOp: {
    const auto& unary = static_cast<const UnaryOpNode&>(node);
    if (!unary.operand) {
        diagnostic("missing unary operand", node.location);
        return;
    }

    const std::string op = text_util::lowercase(unary.op);
    if (op == "+") {
        generateExpression(*unary.operand);
    } else if (op == "-") {
        generateExpression(*unary.operand);
        emit(makeOperation(OperationCode::NEG));
    } else if (op == "not") {
        generateExpression(*unary.operand);
        emit(makeLiteral(RuntimeValue::boolean(false)));
        emit(makeOperation(OperationCode::EQL));
    } else {
        diagnostic("unsupported unary operator '" + unary.op + "'",
node.location);
    }
    break;
}

case ASTNodeKind::BinOp: {
    const auto& binary = static_cast<const BinOpNode&>(node);
    if (!binary.left || !binary.right) {
        diagnostic("missing binary operand", node.location);
        return;
    }

    const std::string op = text_util::lowercase(binary.op);

    if (op == "and") {
        generateExpression(*binary.left);
        int leftFalse = emit(makeJump(OpCode::JPC));
        generateExpression(*binary.right);
        int rightFalse = emit(makeJump(OpCode::JPC));
        emit(makeLiteral(RuntimeValue::boolean(true)));
        int endJump = emit(makeJump(OpCode::JMP));
```

```

        int falseAddress = nextAddress();
        patchOperand(leftFalse, falseAddress);
        patchOperand(rightFalse, falseAddress);
        emit(makeLiteral(RuntimeValue::boolean(false)));
        patchOperand(endJump, nextAddress());
        break;
    }

    if (op == "or") {
        generateExpression(*binary.left);
        int evalRight = emit(makeJump(Opcode::JPC));
        emit(makeLiteral(RuntimeValue::boolean(true)));
        int leftEnd = emit(makeJump(Opcode::JMP));
        patchOperand(evalRight, nextAddress());
        generateExpression(*binary.right);
        int falseJump = emit(makeJump(Opcode::JPC));
        emit(makeLiteral(RuntimeValue::boolean(true)));
        int rightEnd = emit(makeJump(Opcode::JMP));
        patchOperand(falseJump, nextAddress());
        emit(makeLiteral(RuntimeValue::boolean(false)));
        int end = nextAddress();
        patchOperand(leftEnd, end);
        patchOperand(rightEnd, end);
        break;
    }

    OperationCode operation;
    if (!mapBinaryOperator(binary.op, operation)) {
        diagnostic("unsupported binary operator '" + binary.op + "'",
node.location);
        return;
    }

    generateExpression(*binary.left);
    generateExpression(*binary.right);
    emit(makeOperation(operation));
    break;
}

```

Function call

```

case ASTNodeKind::FuncCall: {
    const auto& call = static_cast<const FuncCallNode&>(node);
    const TabEntry* entry = entryAt(symbolTable, node.tabIndex);

```



```

        int functionIndex = node.tabIndex;

        if (!entry && symbolTable) {
            functionIndex = symbolTable->lookupTab(call.name);
            entry = entryAt(symbolTable, functionIndex);
        }

        if (!entry || entry->obj != OBJ_FUNCTION) {
            diagnostic("unknown function '" + call.name + "'",
node.location);
            return;
        }

        const int address = routineAddress(functionIndex);
        if (address < 0) {
            diagnostic("function '" + call.name + "' does not have a
generated entry address", node.location);
            return;
        }

        const std::vector<int> parameterIndices =
routineParameterIndices(entry->ref);
        if (parameterIndices.size() != call.arguments.size()) {
            diagnostic("function '" + call.name + "' argument count does
not match metadata", node.location);
            return;
        }

        for (size_t i = 0; i < call.arguments.size(); ++i) {
            const auto& argument = call.arguments[i];
            if (!argument) {
                diagnostic("function '" + call.name + "' has an empty
argument", node.location);
                return;
            }

            const size_t diagnosticCount = result.diagnostics.size();
            const TabEntry& parameter =
symbolTable->tabAt(parameterIndices[i]);
            emitArgumentValue(*argument, parameter);

            if (result.diagnostics.size() != diagnosticCount) {
                return;
            }
        }
    }

```

```

Instruction instruction;
instruction.opcode = OpCode::CAL;
instruction.level = lexicalLevelOffset(*entry);
instruction.operand = address;
emit(instruction);
break;

```

Load variable / array access / record access

```

bool CodeGenerator::emitLoadAddressable(const ExpressionNode& node) {
    if (node.kind == ASTNodeKind::ArrayAccess ||
        node.kind == ASTNodeKind::RecordAccess) {
        const TypeInfo valueType = expressionTypeInfo(node);
        if (isStructuredType(valueType)) {
            diagnostic("structured value cannot be loaded in this expression
context", node.location);
            return false;
        }

        const size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(node);
        if (result.diagnostics.size() != diagnosticCount) {
            return false;
        }

        Instruction instruction;
        instruction.opcode = OpCode::LDI;
        emit(instruction);
        return true;
    }

    if (node.kind != ASTNodeKind::Var) {
        diagnostic("expression is not addressable", node.location);
        return false;
    }

    const auto& variable = static_cast<const VarNode&>(node);
    const TabEntry* entry = entryAt(symbolTable, node.tabIndex);

    if (!entry && symbolTable) {
        entry = entryAt(symbolTable, symbolTable->lookupTab(variable.name));
    }

    if (!entry) {

```

```

        diagnostic("unknown identifier '" + variable.name + "'",
node.location);
        return false;
    }

    if (entry->obj == OBJ_CONSTANT) {
        return emitConstant(*entry);
    }

    if (isCurrentFunctionResult(*entry)) {
        if (isStructuredType(entry->typeInfo)) {
            diagnostic("structured function result cannot be loaded as a
scalar value", node.location);
            return false;
        }

        Instruction instruction;
        instruction.opcode = OpCode::LOD;
        instruction.level = 0;
        instruction.operand = variableAddress(*entry);
        emit(instruction);
        return true;
    }

    if (entry->obj != OBJ_VARIABLE) {
        diagnostic("identifier '" + variable.name + "' is not a value",
node.location);
        return false;
    }

    if (isStructuredType(entry->typeInfo)) {
        diagnostic("structured value cannot be loaded in this expression
context", node.location);
        return false;
    }

    Instruction instruction;
    instruction.opcode = OpCode::LOD;
    instruction.level = lexicalLevelOffset(*entry);
    instruction.operand = variableAddress(*entry);
    instruction.indirect = entry->nrm == 0;
    emit(instruction);
    return true;
}

```

3.6. Translasi Statement dan Control Flow

Translasi statement pada Milestone 4 dilakukan dengan mengubah setiap statement pada decorated AST menjadi satu atau lebih instruksi intermediate. Proses ini ditangani oleh bagian `codegen_statement`, yang memiliki dispatcher utama untuk mengenali jenis statement, seperti block, assignment, if, while, repeat, for, case, dan procedure call. Setiap jenis statement memiliki pola translasi yang berbeda, tetapi semuanya tetap mengikuti prinsip stack machine, yaitu ekspresi dievaluasi ke operand stack, kemudian hasilnya digunakan oleh instruksi berikutnya.

Assignment scalar diterjemahkan dengan membangkitkan instruksi untuk mengevaluasi ekspresi pada sisi kanan terlebih dahulu. Setelah nilai ekspresi berada di stack, generator membangkitkan instruksi store untuk menyimpan nilai tersebut ke target assignment. Untuk variabel scalar biasa, penyimpanan dilakukan menggunakan STO. Jika target berupa elemen array atau field record, generator menghitung alamat efektif target terlebih dahulu, lalu menggunakan STI untuk menyimpan nilai ke alamat tersebut.

Assignment terhadap array atau record ditangani secara khusus karena array dan record tidak direpresentasikan sebagai satu nilai tunggal, melainkan sebagai beberapa slot memori berurutan. Jika target assignment bertipe array atau record, generator tidak menggunakan store scalar, melainkan membangkitkan alamat source dan alamat destination, kemudian menghasilkan instruksi CPY. Operand pada instruksi CPY berisi ukuran aggregate yang perlu disalin. Dengan mekanisme ini, assignment antar record, antar array, maupun assignment elemen array yang bertipe record dapat dilakukan dengan menyalin seluruh slot memori yang merepresentasikan nilai tersebut.

Statement if diterjemahkan menggunakan instruksi conditional jump. Generator terlebih dahulu membangkitkan kode untuk ekspresi kondisi, sehingga hasil kondisi berada di stack. Setelah itu, generator membangkitkan instruksi JPC yang akan melompat ke bagian else atau akhir statement jika kondisi bernilai false. Jika terdapat bagian else, generator juga membangkitkan instruksi JMP untuk melewati blok else setelah blok then selesai dieksekusi. Karena alamat tujuan jump belum selalu diketahui saat instruksi dibuat, generator menggunakan mekanisme patching untuk mengisi operand instruksi jump setelah alamat target diketahui.

Statement while diterjemahkan dengan menandai alamat awal loop sebelum evaluasi kondisi dilakukan. Generator membangkitkan kode kondisi, kemudian instruksi JPC

untuk keluar dari loop jika kondisi bernilai false. Jika kondisi true, body loop dijalankan, lalu generator membangkitkan instruksi JMP untuk kembali ke alamat awal loop. Setelah body selesai, target dari JPC dipatch ke alamat setelah loop. Dengan pola ini, interpreter dapat menjalankan loop hanya dengan mengikuti instruksi jump tanpa perlu mengetahui struktur while pada source code.

Statement repeat-until memiliki pola translasi yang sedikit berbeda karena body selalu dieksekusi minimal satu kali. Generator terlebih dahulu menyimpan alamat awal body, kemudian membangkitkan instruksi untuk seluruh statement di dalam body. Setelah body selesai, generator membangkitkan kode untuk kondisi until. Jika kondisi bernilai false, instruksi JPC akan mengembalikan eksekusi ke awal body. Jika kondisi bernilai true, eksekusi dilanjutkan ke instruksi setelah loop. Pola ini sesuai dengan sifat repeat-until yang mengevaluasi kondisi di akhir perulangan.

Statement for diterjemahkan dengan menyimpan nilai awal ke control variable, lalu membangkitkan loop yang membandingkan control variable dengan nilai akhir. Untuk for-to, kondisi menggunakan operasi kurang dari atau sama dengan, sedangkan untuk for-downto, kondisi menggunakan operasi lebih dari atau sama dengan. Jika kondisi false, eksekusi keluar dari loop. Jika kondisi true, body dijalankan, kemudian control variable dinaikkan atau diturunkan satu langkah sesuai arah loop. Setelah itu, generator membangkitkan JMP kembali ke awal pengecekan kondisi. Dengan pola ini, for-to dan for-downto dapat ditangani menggunakan struktur intermediate code yang serupa.

Statement case diterjemahkan sebagai serangkaian pengecekan label. Untuk setiap label pada setiap branch, generator membangkitkan kode selector dan kode label, kemudian membandingkannya menggunakan operasi EQL. Jika hasil perbandingan false, eksekusi melompat ke pengecekan label berikutnya. Jika hasil true, statement pada branch tersebut dijalankan, lalu eksekusi melompat ke akhir struktur case. Semua lompatan keluar dari branch disimpan dan dipatch setelah seluruh branch selesai diproses. Dengan pendekatan ini, struktur case dapat direpresentasikan sebagai kombinasi perbandingan dan jump linear.

Procedure call diterjemahkan dengan membangkitkan kode untuk seluruh argumen terlebih dahulu, lalu membangkitkan instruksi CAL. Untuk parameter by-value, nilai argumen dimasukkan ke stack. Untuk parameter by-reference, alamat argumen dimasukkan ke stack agar routine dapat mengubah variabel asli. Jika argumen bertipe

array atau record dan dikirim by-value, generator menggunakan instruksi LDR untuk menyalin seluruh slot aggregate ke stack. Procedure bawaan seperti write, writeln, read, dan readln ditangani secara khusus. write dan writeln diterjemahkan menjadi operasi output melalui OPR, sedangkan read dan readln diterjemahkan menjadi instruksi input INP dan INL.

Dispatcher statement + assignment scalar/aggregate

```
void CodeGenerator::generateStatement(const StatementNode& node) {
    switch (node.kind) {
        case ASTNodeKind::Block:
            generateBlock(static_cast<const BlockNode&>(node));
            break;
        case ASTNodeKind::Assign:
            generateAssignment(static_cast<const AssignNode&>(node));
            break;
        case ASTNodeKind::If:
            generateIf(static_cast<const IfNode&>(node));
            break;
        case ASTNodeKind::While:
            generateWhile(static_cast<const WhileNode&>(node));
            break;
        case ASTNodeKind::Repeat:
            generateRepeat(static_cast<const RepeatNode&>(node));
            break;
        case ASTNodeKind::For:
            generateFor(static_cast<const ForNode&>(node));
            break;
        case ASTNodeKind::Case:
            generateCase(static_cast<const CaseNode&>(node));
            break;
        case ASTNodeKind::ProcCall:
            generateProcedureCall(static_cast<const ProcCallNode&>(node));
            break;
        default:
            diagnostic("unsupported statement node for code generation",
node.location);
            break;
    }
}

void CodeGenerator::generateAssignment(const AssignNode& node) {
    if (!node.target || !node.value) {
        diagnostic("assignment is missing its target or value",
```

```

node.location);
    return;
}

const TypeInfo targetType = expressionTypeInfo(*node.target);
if (isStructuredType(targetType)) {
    if (!symbolTable) {
        diagnostic("missing symbol table for structured assignment",
node.location);
        return;
    }

    const TypeInfo valueType = expressionTypeInfo(*node.value);
    if (!isStructuredType(valueType)) {
        diagnostic("structured assignment source is not an array or
record", node.value->location);
        return;
    }

    const int copySize = symbolTable->sizeof(targetType);
    if (copySize <= 0) {
        diagnostic("structured assignment has invalid copy size",
node.location);
        return;
    }

    size_t diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*node.value);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*node.target);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    Instruction instruction;
    instruction.opcode = OpCode::CPY;
    instruction.operand = copySize;
    emit(instruction);
    return;
}

const size_t diagnosticCount = result.diagnostics.size();

```

```

generateExpression(*node.value);
if (result.diagnostics.size() != diagnosticCount) {
    return;
}

emitStoreAddressable(*node.target);
}

```

Store scalar, array element, dan record field

```

bool CodeGenerator::emitStoreAddressable(const ExpressionNode& node) {
    if (node.kind == ASTNodeKind::ArrayAccess ||
        node.kind == ASTNodeKind::RecordAccess) {
        const TypeInfo targetType = expressionTypeInfo(node);
        if (isStructuredType(targetType)) {
            diagnostic("structured target cannot be stored with a scalar
store", node.location);
            return false;
        }

        const size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(node);
        if (result.diagnostics.size() != diagnosticCount) {
            return false;
        }

        Instruction instruction;
        instruction.opcode = OpCode::STI;
        emit(instruction);
        return true;
    }

    if (node.kind != ASTNodeKind::Var) {
        diagnostic("assignment target is not addressable", node.location);
        return false;
    }

    const auto& variable = static_cast<const VarNode&>(node);
    const TabEntry* entry = resolveVariableEntry(symbolTable, variable,
node.tabIndex);
    if (!entry) {
        diagnostic("unknown identifier '" + variable.name + "'",
node.location);
        return false;
    }
}

```



```

    }

    if (entry->obj != OBJ_VARIABLE) {
        if (!isCurrentFunctionResult(*entry)) {
            diagnostic("identifier '" + variable.name + "' is not a variable",
node.location);
            return false;
        }
    }

    if (isStructuredType(entry->typeInfo)) {
        diagnostic("structured target cannot be stored with a scalar store",
node.location);
        return false;
    }

    Instruction instruction;
    instruction.opcode = OpCode::STO;
        instruction.level = isCurrentFunctionResult(*entry) ? 0 :
lexicalLevelOffset(*entry);
    instruction.operand = variableAddress(*entry);
        instruction.indirect = entry->nrm == 0 &&
!isCurrentFunctionResult(*entry);
    emit(instruction);
    return true;
}

```

Translasi if dan while

```

void CodeGenerator::generateIf(const IfNode& node) {
    if (!node.condition || !node.thenBranch) {
        diagnostic("if statement is missing its condition or then-branch",
node.location);
        return;
    }

    const size_t diagnosticCount = result.diagnostics.size();
    generateExpression(*node.condition);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    const int falseJump = emit(makeJump(OpCode::JPC));
    generateStatement(*node.thenBranch);
}

```

```

        if (node.elseBranch) {
            const int endJump = emit(makeJump(OpCode::JMP));
            patchOperand(falseJump, nextAddress());
            generateStatement(*node.elseBranch);
            patchOperand(endJump, nextAddress());
        } else {
            patchOperand(falseJump, nextAddress());
        }
    }
}

void CodeGenerator::generateWhile(const WhileNode& node) {
    if (!node.condition || !node.body) {
        diagnostic("while statement is missing its condition or body",
node.location);
        return;
    }

    const int loopStart = nextAddress();
    const size_t diagnosticCount = result.diagnostics.size();
    generateExpression(*node.condition);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    const int exitJump = emit(makeJump(OpCode::JPC));
    generateBlock(*node.body);
    emit(makeJump(OpCode::JMP, loopStart));
    patchOperand(exitJump, nextAddress());
}

```

Translasi repeat dan for

```

void CodeGenerator::generateRepeat(const RepeatNode& node) {
    if (!node.condition) {
        diagnostic("repeat-until statement is missing its condition",
node.location);
        return;
    }

    const int loopStart = nextAddress();
    for (const auto& statement : node.body) {
        if (!statement) {
            continue;
        }
    }
}

```

```

    }

    generateStatement(*statement);
}

const size_t diagnosticCount = result.diagnostics.size();
generateExpression(*node.condition);
if (result.diagnostics.size() != diagnosticCount) {
    return;
}

emit(makeJump(OpCode::JPC, loopStart));
}

void CodeGenerator::generateFor(const ForNode& node) {
    if (!symbolTable) {
        diagnostic("missing symbol table for for-statement code generation",
node.location);
        return;
    }

    if (!node.startValue || !node.endValue || !node.body) {
        diagnostic("for statement is missing its bounds or body",
node.location);
        return;
    }

    const int index = symbolTable->lookupTab(node.controlVariable);
    if (index == 0) {
        diagnostic("unknown control variable '" + node.controlVariable + "'",
node.location);
        return;
    }

    const TabEntry& control = symbolTable->tabAt(index);
    if (control.obj != OBJ_VARIABLE) {
        diagnostic("for control identifier '" + node.controlVariable + "' is
not a variable", node.location);
        return;
    }

    size_t diagnosticCount = result.diagnostics.size();
    generateExpression(*node.startValue);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }
}

```

```

Instruction initStore;
initStore.opcode = OpCode::STO;
initStore.level = lexicalLevelOffset(control);
initStore.operand = variableAddress(control);
initStore.indirect = control.nrm == 0;
emit(initStore);

const int loopStart = nextAddress();

Instruction loadControl;
loadControl.opcode = OpCode::LOD;
loadControl.level = lexicalLevelOffset(control);
loadControl.operand = variableAddress(control);
loadControl.indirect = control.nrm == 0;
emit(loadControl);

diagnosticCount = result.diagnostics.size();
generateExpression(*node.endValue);
if (result.diagnostics.size() != diagnosticCount) {
    return;
}

emit(makeOperation(
    node.direction == ForDirection::Downto ? OperationCode::GEQ :
OperationCode::LEQ
));

const int exitJump = emit(makeJump(OpCode::JPC));
generateBlock(*node.body);

loadControl.operand = variableAddress(control);
loadControl.level = lexicalLevelOffset(control);
emit(loadControl);
emit(makeLiteral(RuntimeValue::integer(1)));
emit(makeOperation(
    node.direction == ForDirection::Downto ? OperationCode::SUB :
OperationCode::ADD
));
emit(initStore);
emit(makeJump(OpCode::JMP, loopStart));

patchOperand(exitJump, nextAddress());
}

```

Translasi case

```
void CodeGenerator::generateCase(const CaseNode& node) {
    if (!node.selector) {
        diagnostic("case statement is missing its selector", node.location);
        return;
    }

    std::vector<int> endJumps;

    for (const CaseBranchNode& branch : node.branches) {
        if (!branch.statement) {
            diagnostic("case branch is missing its statement",
branch.location);
            continue;
        }

        if (branch.labels.empty()) {
            diagnostic("case branch has no labels", branch.location);
            continue;
        }

        for (const auto& label : branch.labels) {
            if (!label) {
                diagnostic("case branch has an empty label", branch.location);
                continue;
            }

            const size_t diagnosticCount = result.diagnostics.size();
            generateExpression(*node.selector);
            generateExpression(*label);
            if (result.diagnostics.size() != diagnosticCount) {
                return;
            }

            emit(makeOperation(OperationCode::EQL));
            const int nextCheck = emit(makeJump(OpCode::JPC));
            generateStatement(*branch.statement);
            endJumps.push_back(emit(makeJump(OpCode::JMP)));
            patchOperand(nextCheck, nextAddress());
        }
    }

    const int endAddress = nextAddress();
    for (int jumpIndex : endJumps) {
        patchOperand(jumpIndex, endAddress);
    }
}
```

```

    }
}

```

3.7. Implementasi Procedure, Function, dan Stack Frame

Procedure dan function pada program kami diimplementasikan menggunakan mekanisme stack frame. Setiap routine memiliki metadata yang menyimpan alamat awal routine, jumlah parameter, ukuran frame, informasi apakah routine mengembalikan nilai, serta offset slot return value. Metadata ini digunakan oleh interpreter ketika instruksi CAL dijalankan untuk membentuk frame baru dan ketika instruksi RET dijalankan untuk mengembalikan eksekusi ke frame pemanggil.

Pada saat procedure atau function dipanggil, argumen terlebih dahulu diletakkan pada operand stack. Instruksi CAL kemudian membuat frame baru di memori. Frame tersebut terdiri atas static link, dynamic link, return address, parameter, variabel lokal, dan slot return value khusus untuk function scalar. Static link digunakan untuk mengakses scope luar pada nested routine, sedangkan dynamic link digunakan untuk kembali ke frame pemanggil. Return address menyimpan alamat instruksi yang harus dijalankan setelah routine selesai.

Untuk function, nilai return disimpan pada slot khusus di frame function. Ketika RET dijalankan, interpreter mengambil nilai dari slot return tersebut, mengembalikan base dan ip ke frame pemanggil, lalu meletakkan nilai return ke operand stack. Dengan mekanisme ini, function call dapat digunakan sebagai bagian dari ekspresi, sedangkan procedure call cukup menjalankan instruksi tanpa menghasilkan nilai.

RoutineMetadata

```

struct RoutineMetadata {
    int address = 0;
    int parameterCount = 0;
    int frameSize = 0;
    bool returnsValue = false;
    int returnValueOffset = 0;
    std::vector<bool> byReferenceParameters;
};

```

```

struct IntermediateProgram {
    std::vector<Instruction> instructions;
    std::vector<RoutineMetadata> routines;
};

std::string opcodeName(OpCode opcode);
std::string operationName(OperationCode operation);
std::string formatInstruction(const Instruction& instruction, int address);
std::vector<std::string> formatProgram(const IntermediateProgram& program);

```

registerRoutine

```

void CodeGenerator::registerRoutine(
    int tabIndex,
    int address,
    const BTabEntry& block,
    bool returnsValue
) {
    if (tabIndex <= 0 || tabIndex >=
static_cast<int>(routineAddressesByTabIndex.size())) {
        diagnostic("cannot register routine with invalid tab index " +
std::to_string(tabIndex));
        return;
    }

    const int returnValueOffset = frameHeaderSize + block.psize + block.vsize;
    RoutineMetadata metadata;
    metadata.address = address;
    metadata.parameterCount = block.psize;
    metadata.frameSize = frameHeaderSize + block.psize + block.vsize +
(returnsValue ? 1 : 0);
    metadata.returnsValue = returnsValue;
    metadata.returnValueOffset = returnsValue ? returnValueOffset : 0;

    for (int parameterIndex :
routineParameterIndices(symbolTable->tabAt(tabIndex).ref)) {

metadata.byReferenceParameters.push_back(symbolTable->tabAt(parameterIndex).nr
m == 0);
    }

    routineAddressesByTabIndex[static_cast<size_t>(tabIndex)] = address;
    result.program.routines.push_back(metadata);
}

```

procedure declaration

```
void CodeGenerator::generateProcDecl(const ProcDeclNode& node) {
    if (!symbolTable) {
        diagnostic("missing symbol table for procedure declaration",
node.location);
        return;
    }

    if (node.blockIndex <= 0 || node.blockIndex >=
static_cast<int>(symbolTable->btabs().size())) {
        diagnostic("procedure block index is out of range for '" + node.name +
"', node.location);
        return;
    }

    const BTabEntry& block = symbolTable->btabsAt(node.blockIndex);
    const int address = nextAddress();
    registerRoutine(node.tabIndex, address, block, false);

    const int savedBlock = currentBlockIndex;
    const int savedLevel = currentLexLevel;
    const int savedFunctionTab = currentFunctionTabIndex;
    const int savedFunctionReturnOffset = currentFunctionReturnOffset;

    currentBlockIndex = node.blockIndex;
    currentLexLevel = node.lexLevel + 1;
    currentFunctionTabIndex = 0;
    currentFunctionReturnOffset = 0;

    Instruction init;
    init.opcode = OpCode::INT;
    init.operand = frameHeaderSize + block.psize + block.vsize;
    emit(init);

    int bodyJump = -1;
    if (hasRoutineDeclaration(node.declarations)) {
        Instruction jump;
        jump.opcode = OpCode::JMP;
        bodyJump = emit(jump);
    }

    for (const auto& declaration : node.declarations) {
        if (declaration) {
```



```

        generateDeclaration(*declaration);
    }
}

if (bodyJump >= 0) {
    patchOperand(bodyJump, nextAddress());
}

if (node.body) {
    generateBlock(*node.body);
} else {
    diagnostic("procedure '" + node.name + "' is missing its body",
node.location);
}

Instruction ret;
ret.opcode = OpCode::RET;
emit(ret);

currentBlockIndex = savedBlock;
currentLexLevel = savedLevel;
currentFunctionTabIndex = savedFunctionTab;
currentFunctionReturnOffset = savedFunctionReturnOffset;
}

```

function declaration

```

void CodeGenerator::generateFuncDecl(const FuncDeclNode& node) {
    if (!symbolTable) {
        diagnostic("missing symbol table for function declaration",
node.location);
        return;
    }

    if (node.blockIndex <= 0 || node.blockIndex >=
static_cast<int>(symbolTable->btab().size())) {
        diagnostic("function block index is out of range for '" + node.name +
"'", node.location);
        return;
    }

    const BTabEntry& block = symbolTable->btabAt(node.blockIndex);
    const int address = nextAddress();
    registerRoutine(node.tabIndex, address, block, true);
}

```

```

const int savedBlock = currentBlockIndex;
const int savedLevel = currentLexLevel;
const int savedFunctionTab = currentFunctionTabIndex;
const int savedFunctionReturnOffset = currentFunctionReturnOffset;

currentBlockIndex = node.blockIndex;
currentLexLevel = node.lexLevel + 1;
currentFunctionTabIndex = node.tabIndex;
currentFunctionReturnOffset = frameHeaderSize + block.psize + block.vsize;

Instruction init;
init.opcode = OpCode::INT;
init.operand = frameHeaderSize + block.psize + block.vsize + 1;
emit(init);

int bodyJump = -1;
if (hasRoutineDeclaration(node.declarations)) {
    Instruction jump;
    jump.opcode = OpCode::JMP;
    bodyJump = emit(jump);
}

for (const auto& declaration : node.declarations) {
    if (declaration) {
        generateDeclaration(*declaration);
    }
}

if (bodyJump >= 0) {
    patchOperand(bodyJump, nextAddress());
}

if (node.body) {
    generateBlock(*node.body);
} else {
    diagnostic("function '" + node.name + "' is missing its body",
node.location);
}

Instruction ret;
ret.opcode = OpCode::RET;
emit(ret);

currentBlockIndex = savedBlock;
currentLexLevel = savedLevel;

```

```
currentFunctionTabIndex = savedFunctionTab;
currentFunctionReturnOffset = savedFunctionReturnOffset;}
```

3.8. Implementasi Array, Record, dan Parameter Terstruktur

Array dan record pada program kami direpresentasikan sebagai beberapa slot memori berurutan. Array menggunakan informasi pada atab, seperti batas bawah, ukuran elemen, dan ukuran total array. Record menggunakan informasi offset field yang tersimpan pada symbol table. Karena array dan record tidak dapat direpresentasikan sebagai satu nilai scalar, akses terhadap keduanya dilakukan melalui perhitungan alamat efektif.

Untuk akses array, generator terlebih dahulu mengambil alamat dasar array, kemudian menghitung offset berdasarkan indeks, batas bawah, dan ukuran elemen. Untuk akses record, generator mengambil alamat dasar record lalu menambahkan offset field. Setelah alamat efektif diperoleh, pembacaan nilai dilakukan dengan LDI, sedangkan penulisan nilai dilakukan dengan STI. Pola ini memungkinkan akses terhadap elemen array, field record, dan kombinasi seperti array of record.

Assignment terhadap array atau record ditangani menggunakan instruksi CPY. Generator membangkitkan alamat source dan destination, lalu interpreter menyalin sejumlah slot sesuai ukuran aggregate. Selain itu, parameter array dan record juga didukung. Untuk parameter by-reference, alamat aggregate dikirim ke routine. Untuk parameter by-value, generator menggunakan instruksi LDR agar seluruh slot aggregate disalin ke stack, sehingga perubahan di dalam routine tidak memengaruhi variabel asli pemanggil.

assignment aggregate CPY dan scalar component STI

```
void CodeGenerator::generateAssignment(const AssignNode& node) {
    if (!node.target || !node.value) {
        diagnostic("assignment is missing its target or value",
node.location);
        return;
    }

    const TypeInfo targetType = expressionTypeInfo(*node.target);
    if (isStructuredType(targetType)) {
        if (!symbolTable) {
            diagnostic("missing symbol table for structured assignment",
node.location);
            return;
        }
    }
}
```

```

    }

    const TypeInfo valueType = expressionTypeInfo(*node.value);
    if (!isStructuredType(valueType)) {
        diagnostic("structured assignment source is not an array or
record", node.value->location);
        return;
    }

    const int copySize = symbolTable->sizeof(targetType);
    if (copySize <= 0) {
        diagnostic("structured assignment has invalid copy size",
node.location);
        return;
    }

    size_t diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*node.value);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*node.target);
    if (result.diagnostics.size() != diagnosticCount) {
        return;
    }

    Instruction instruction;
    instruction.opcode = OpCode::CPY;
    instruction.operand = copySize;
    emit(instruction);
    return;
}

const size_t diagnosticCount = result.diagnostics.size();
generateExpression(*node.value);
if (result.diagnostics.size() != diagnosticCount) {
    return;
}

emitStoreAddressable(*node.target);
}

bool CodeGenerator::emitStoreAddressable(const ExpressionNode& node) {
    if (node.kind == ASTNodeKind::ArrayAccess ||

```

```

        node.kind == ASTNodeKind::RecordAccess) {
            const TypeInfo targetType = expressionTypeInfo(node);
            if (isStructuredType(targetType)) {
                diagnostic("structured target cannot be stored with a scalar
store", node.location);
                return false;
            }

            const size_t diagnosticCount = result.diagnostics.size();
            emitAddressAddressable(node);
            if (result.diagnostics.size() != diagnosticCount) {
                return false;
            }

            Instruction instruction;
            instruction.opcode = OpCode::STI;
            emit(instruction);
            return true;
        }
}

```

perhitungan alamat array/record

```

bool CodeGenerator::emitAddressAddressable(const ExpressionNode& node) {
    if (node.kind == ASTNodeKind::ArrayAccess) {
        const auto& access = static_cast<const ArrayAccessNode&>(node);
        if (!access.array) {
            diagnostic("array access is missing its base expression",
node.location);
            return false;
        }

        TypeInfo currentType = expressionTypeInfo(*access.array);
        const size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(*access.array);
        if (result.diagnostics.size() != diagnosticCount) {
            return false;
        }

        for (const auto& index : access.indices) {
            if (!index) {
                diagnostic("array access has an empty index expression",
node.location);
                return false;
            }
        }
    }
}

```

```

        if (currentType.code != TYPE_ARRAY ||
            currentType.ref <= 0 ||
            currentType.ref >=
static_cast<int>(symbolTable->atab().size())) {
            diagnostic("subscripted expression is not an array",
node.location);
            return false;
        }

        const ATabEntry& arrayEntry =
symbolTable->atabAt(currentType.ref);
        generateExpression(*index);
        if (result.diagnostics.size() != diagnosticCount) {
            return false;
        }

        if (arrayEntry.low != 0) {
            emit(makeLiteral(RuntimeValue::integer(arrayEntry.low)));
            emit(makeOperation(OperationCode::SUB));
        }

        if (arrayEntry.elsz != 1) {
            emit(makeLiteral(RuntimeValue::integer(arrayEntry.elsz)));
            emit(makeOperation(OperationCode::MUL));
        }

        emit(makeOperation(OperationCode::ADD));
        currentType = arrayElementType(arrayEntry);
    }

    return true;
}

if (node.kind == ASTNodeKind::RecordAccess) {
    const auto& access = static_cast<const RecordAccessNode&>(node);
    if (!access.record) {
        diagnostic("record access is missing its base expression",
node.location);
        return false;
    }

    TypeInfo recordType = expressionTypeInfo(*access.record);
    if (recordType.code != TYPE_RECORD || recordType.ref <= 0) {
        diagnostic("field access requires a record operand",
node.location);
    }
}

```

```

        return false;
    }

    const int fieldIndex = symbolTable ?
symbolTable->lookupTab(access.fieldName, recordType.ref) : 0;
    if (fieldIndex <= 0) {
        diagnostic("record type does not contain field '" +
access.fieldName + "'", node.location);
        return false;
    }

    const size_t diagnosticCount = result.diagnostics.size();
    emitAddressAddressable(*access.record);
    if (result.diagnostics.size() != diagnosticCount) {
        return false;
    }

    const TabEntry& field = symbolTable->tabAt(fieldIndex);
    if (field.adr != 0) {
        emit(makeLiteral(RuntimeValue::integer(field.adr)));
        emit(makeOperation(OperationCode::ADD));
    }

    return true;
}

if (node.kind != ASTNodeKind::Var) {
    diagnostic("expression is not addressable", node.location);
    return false;
}

const auto& variable = static_cast<const VarNode&>(node);
const TabEntry* entry = entryAt(symbolTable, node.tabIndex);

if (!entry && symbolTable) {
    entry = entryAt(symbolTable, symbolTable->lookupTab(variable.name));
}

if (!entry) {
    diagnostic("unknown identifier '" + variable.name + "'",
node.location);
    return false;
}

if (entry->obj != OBJ_VARIABLE) {
    diagnostic("identifier '" + variable.name + "' is not addressable",

```

```

node.location);
    return false;
}

Instruction instruction;
if (entry->nrm == 0) {
    instruction.opcode = OpCode::LOD;
    instruction.level = lexicalLevelOffset(*entry);
    instruction.operand = variableAddress(*entry);
} else {
    instruction.opcode = OpCode::LDA;
    instruction.level = lexicalLevelOffset(*entry);
    instruction.operand = variableAddress(*entry);
}

emit(instruction);
return true;
}

```

parameter aggregate by-value dengan LDR

```

bool CodeGenerator::emitArgumentValue(const ExpressionNode& argument, const
TabEntry& parameter) {
    if (parameter.nrm == 0) {
        return emitAddressAddressable(argument);
    }

    if (isStructuredType(parameter.typeInfo)) {
        if (!symbolTable) {
            diagnostic("missing symbol table for aggregate argument",
argument.location);
            return false;
        }

        const int argumentSize = symbolTable->sizeof(parameter.typeInfo);
        if (argumentSize <= 0) {
            diagnostic("aggregate argument has invalid size",
argument.location);
            return false;
        }

        const size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(argument);
        if (result.diagnostics.size() != diagnosticCount) {

```



```

        return false;
    }

    Instruction instruction;
    instruction.opcode = OpCode::LDR;
    instruction.operand = argumentSize;
    emit(instruction);
    return true;
}

generateExpression(argument);
return true;
}

```

3.9. Implementasi Stack Machine Interpreter

Stack machine interpreter merupakan komponen yang menjalankan intermediate code hasil code generator. Interpreter bekerja dengan menyimpan daftar instruksi dalam IntermediateProgram, lalu menjalankan instruksi secara berurutan menggunakan instruction pointer. Setiap instruksi dibaca pada tahap fetch, dikenali pada tahap decode, lalu dijalankan pada tahap execute.

Interpreter memiliki beberapa struktur runtime utama. memory digunakan untuk menyimpan frame, parameter, variabel, dan nilai return. stack digunakan sebagai operand stack untuk evaluasi ekspresi. ip menyimpan alamat instruksi yang sedang berjalan, sedangkan base menunjuk frame aktif. Selain itu, terdapat routineStack untuk mencatat routine yang sedang dieksekusi dan ExecutionResult untuk menyimpan output serta runtime diagnostics.

Instruksi seperti INT, LIT, LOD, STO, JMP, JPC, CAL, RET, dan OPR dieksekusi melalui dispatcher pada executeInstruction. Instruksi OPR meneruskan eksekusi ke fungsi operasi seperti arithmetic, comparison, dan output. Jika terjadi kesalahan runtime, interpreter mencatat diagnostic dan menghentikan eksekusi secara terkendali.

struktur class interpreter

```

class StackMachineInterpreter {
public:
    ExecutionResult execute(const IntermediateProgram& program);

```

```

private:
    void reset();
    bool executeInstruction(const IntermediateProgram& program, const
Instruction& instruction);

    RuntimeValue pop();
    void push(RuntimeValue value);
    RuntimeValue load(int level, int address, bool indirect);
    void store(int level, int address, RuntimeValue value, bool indirect);
    int resolveBase(int level);
    int resolveAddress(int level, int address, bool indirect);
    int popAddress(const std::string& instructionName);
    const RoutineMetadata* findRoutine(const IntermediateProgram& program, int
address) const;
    bool readInputToken(std::string& token);
    void discardInputLine(bool consumeWhenNoBuffered);
    RuntimeValue parseInputValue(const std::string& token, int typeCode);

    bool executeOperation(OperationCode operation);
    bool executeBinaryNumeric(OperationCode operation);
    bool executeComparison(OperationCode operation);

    void runtimeError(const std::string& message);

    std::vector<RuntimeValue> memory;
    std::vector<RuntimeValue> stack;
    std::vector<int> routineStack;
    std::vector<std::string> inputTokens;
    std::string currentOutput;
    ExecutionResult result;
    size_t nextInputToken = 0;
    int ip = 0;
    int base = 0;
};

```

dispatcher instruksi utama

```

bool StackMachineInterpreter::executeInstruction(
    const IntermediateProgram& program,
    const Instruction& instruction) {
    switch (instruction.opcode) {
        // INT m
        case OpCode::INT: {
            const int size = instruction.operand;

```

```

        if (size < 0) {
            runtimeError("INT: ukuran memori tidak valid: " +
std::to_string(size));
            return false;
        }
        const int requiredSize = base + size;
        if (static_cast<int>(memory.size()) < requiredSize) {
            memory.resize(static_cast<size_t>(requiredSize));
        }
        break;
    }
    // LIT v
    case OpCode::LIT: {
        if (instruction.hasLiteral) {
            push(instruction.literal);
        } else {
            // Fallback: jika tidak ada literal, perlakukan operand
sebagai integer.
            push(RuntimeValue::integer(instruction.operand));
        }
        break;
    }
    // LOD a
    case OpCode::LOD: {
        RuntimeValue value = load(instruction.level, instruction.operand,
instruction.indirect);
        if (result.halted) return false;
        push(std::move(value));
        break;
    }
    // LDA a
    case OpCode::LDA: {
        const int absoluteAddress = resolveAddress(instruction.level,
instruction.operand, false);
        if (result.halted) return false;
        push(RuntimeValue::integer(absoluteAddress));
        break;
    }
    // LDI
    case OpCode::LDI: {
        const int absoluteAddress = popAddress("LDI");
        if (result.halted) return false;
        push(memory[static_cast<size_t>(absoluteAddress)]);
        break;
    }
    // LDR n

```

```

        case OpCode::LDR: {
            const int size = instruction.operand;
            if (size < 0) {
                runtimeError("LDR: ukuran load tidak valid: " +
std::to_string(size));
                return false;
            }

            const int sourceAddress = popAddress("LDR");
            if (result.halted) return false;

            if (sourceAddress + size > static_cast<int>(memory.size())) {
                runtimeError("LDR: out-of-bounds aggregate load");
                return false;
            }

            for (int offset = 0; offset < size; ++offset) {
                push(memory[static_cast<size_t>(sourceAddress + offset)]);
            }
            break;
        }
// STO a
        case OpCode::STO: {
            RuntimeValue value = pop();
            if (result.halted) return false;
            store(instruction.level, instruction.operand, std::move(value),
instruction.indirect);
            break;
        }
// STI
        case OpCode::STI: {
            const int absoluteAddress = popAddress("STI");
            if (result.halted) return false;
            RuntimeValue value = pop();
            if (result.halted) return false;
            memory[static_cast<size_t>(absoluteAddress)] = std::move(value);
            break;
        }
// CPY n
        case OpCode::CPY: {
            const int size = instruction.operand;
            if (size < 0) {
                runtimeError("CPY: ukuran copy tidak valid: " +
std::to_string(size));
                return false;
            }

```

```

        const int destinationAddress = popAddress("CPY");
        if (result.halted) return false;
        const int sourceAddress = popAddress("CPY");
        if (result.halted) return false;

        if (sourceAddress + size > static_cast<int>(memory.size()) ||
            destinationAddress + size > static_cast<int>(memory.size())) {
            runtimeError("CPY: out-of-bounds aggregate copy");
            return false;
        }

        std::vector<RuntimeValue> copiedValues;
        copiedValues.reserve(static_cast<size_t>(size));
        for (int offset = 0; offset < size; ++offset) {
            copiedValues.push_back(memory[static_cast<size_t>(sourceAddress + offset)]);
        }

        for (int offset = 0; offset < size; ++offset) {
            memory[static_cast<size_t>(destinationAddress + offset)] =
                std::move(copiedValues[static_cast<size_t>(offset)]);
        }
        break;
    }
    // INP type
    case OpCode::INP: {
        const int absoluteAddress = popAddress("INP");
        if (result.halted) return false;

        std::string token;
        if (!readInputToken(token)) {
            return false;
        }

        RuntimeValue value = parseInputValue(token, instruction.operand);
        if (result.halted) return false;
        memory[static_cast<size_t>(absoluteAddress)] = std::move(value);
        break;
    }
}

```

siklus fetch, decode, execute

```

ExecutionResult StackMachineInterpreter::execute(const IntermediateProgram&
program) {

```

```

    reset();
    const int programSize = static_cast<int>(program.instructions.size());
    if (programSize == 0) {
        result.diagnostics.push_back({"program kosong: tidak ada instruksi",
-1});
        result.halted = true;
        return result;
    }
    while (ip >= 0 && ip < programSize && !result.halted) {
        // Fetch
        const Instruction& instruction =
program.instructions[static_cast<size_t>(ip)];
        ++ip;
        // Decode + Execute
        executeInstruction(program, instruction);
    }
    if (!result.halted) {
        result.diagnostics.push_back({
            "Runtime error: instruction pointer left program without RET at "
+ std::to_string(ip),
            ip
        });
        result.halted = true;
    }
    return result;
}

```

3.10. Implementasi Input dan Output

Input dan output pada Milestone 4 diimplementasikan sebagai bagian dari intermediate code. Procedure bawaan write dan writeln diterjemahkan oleh code generator menjadi instruksi OPR dengan operasi WRT atau WRTLN. Saat interpreter menjalankan operasi tersebut, nilai diambil dari operand stack dan ditambahkan ke output program. Perbedaan utama write dan writeln adalah writeln menutup baris output setelah nilai terakhir dicetak.

Procedure bawaan read dan readln diterjemahkan menjadi instruksi INP dan INL. Untuk setiap argumen input, generator membangkitkan alamat target terlebih dahulu, lalu menghasilkan instruksi INP dengan operand berupa kode tipe target. Interpreter membaca token dari input pengguna, mengonversinya menjadi RuntimeValue sesuai tipe target, lalu menyimpannya ke alamat memori tersebut.

Instruksi `readln` memiliki perilaku tambahan melalui `INL`. Jika masih ada token tersisa pada baris input, token tersebut dibuang. Jika tidak ada token tersisa dan `readln` dipanggil tanpa argumen, interpreter membaca satu baris untuk dilewati.

translasi `write`, `writeln`, `read`, dan `readln`

```
void CodeGenerator::generateProcedureCall(const ProcCallNode& node) {
    const std::string lowered = text_util::lowercase(node.name);

    if (isBuiltinOutput(lowered)) {
        if (node.arguments.empty()) {
            if (lowered == "writeln") {
                emit(makeLiteral(RuntimeValue::string("")));
                emit(makeOperation(OperationCode::WRTLN));
            }
            return;
        }

        for (size_t i = 0; i < node.arguments.size(); ++i) {
            const auto& argument = node.arguments[i];
            if (!argument) {
                diagnostic("procedure '" + node.name + "' has an empty
argument", node.location);
                return;
            }

            const size_t diagnosticCount = result.diagnostics.size();
            generateExpression(*argument);
            if (result.diagnostics.size() != diagnosticCount) {
                return;
            }

            const bool isLast = i + 1 == node.arguments.size();
            emit(makeOperation(
                lowered == "writeln" && isLast ? OperationCode::WRTLN :
OperationCode::WRT
            ));
        }
        return;
    }

    if (isBuiltinInput(lowered)) {
        for (const auto& argument : node.arguments) {
```

```

        if (!argument) {
            diagnostic("procedure '" + node.name + "' has an empty
argument", node.location);
            return;
        }

        const TypeInfo targetType = expressionTypeInfo(*argument);
        if (isStructuredType(targetType)) {
            diagnostic("input procedures do not support structured
targets", argument->location);
            return;
        }

        const size_t diagnosticCount = result.diagnostics.size();
        emitAddressAddressable(*argument);
        if (result.diagnostics.size() != diagnosticCount) {
            return;
        }

        Instruction instruction;
        instruction.opcode = OpCode::INP;
        instruction.operand = targetType.code;
        emit(instruction);
    }

    if (lowered == "readln") {
        Instruction instruction;
        instruction.opcode = OpCode::INL;
        instruction.operand = node.arguments.empty() ? 1 : 0;
        emit(instruction);
    }
    return;
}

```

3.11. Error Handling dan Diagnostic

Error handling pada Milestone 4 dilakukan secara bertahap sesuai pipeline compiler. Jika semantic analyzer menemukan error, proses code generation tidak dijalankan. Jika code generator menghasilkan diagnostic, interpreter tidak dijalankan. Jika semantic analysis dan code generation berhasil, barulah interpreter menjalankan intermediate code. Dengan alur ini, setiap kesalahan ditangani pada tahap yang tepat dan tidak menyebabkan program crash.

Semantic diagnostics digunakan untuk kesalahan makna program, seperti tipe yang tidak sesuai, identifier yang tidak valid, atau penggunaan structured value pada konteks yang tidak didukung. Codegen diagnostics digunakan untuk kesalahan pada proses pembangkitan intermediate code, misalnya node yang tidak dapat diterjemahkan atau metadata yang tidak konsisten. Runtime diagnostics digunakan untuk kesalahan saat eksekusi, seperti division by zero, stack underflow, invalid jump target, akses memori di luar batas, atau input yang tidak sesuai tipe target.

BAB IV

PENGUJIAN

4.1. Pengujian 1

Input
<pre>test/milestone-4/input/1.txt program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Output
<pre>test/milestone-4/output/1.txt Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 INT 0 5 1 LIT 0 5 2 STO 0 3 3 LOD 0 3 4 LIT 0 10 5 OPR 0 2 6 STO 0 4 7 LIT 0 "Result = " 8 OPR 0 13 9 LOD 0 4 10 OPR 0 14 11 RET 0 0 Codegen diagnostics: No code generation errors.</pre>

Program Output:

Result = 15

Runtime diagnostics:

No runtime errors.

Status: SUCCESS

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 1.txt
```

```
Intermediate code and interpreter result:
```

```
Semantic diagnostics:
```

```
No semantic errors or warnings.
```

```
Intermediate Code:
```

```
0 INT 0 5
1 LIT 0 5
2 STO 0 3
3 LOD 0 3
4 LIT 0 10
5 OPR 0 2
6 STO 0 4
7 LIT 0 "Result = "
8 OPR 0 13
9 LOD 0 4
10 OPR 0 14
11 RET 0 0
```

```
Codegen diagnostics:
```

```
No code generation errors.
```

```
Program Output:
```

```
Result = 15
```

```
Runtime diagnostics:
```

```
No runtime errors.
```

```
Status: SUCCESS
```

Penjelasan

Pengujian ini bertujuan untuk memastikan bahwa program dapat menangani struktur dasar bahasa Arion, mulai dari deklarasi variabel, assignment, operasi aritmetika sederhana, hingga pencetakan keluaran menggunakan `writeln`. Input pada test case ini merepresentasikan program sederhana yang tidak melibatkan struktur kontrol kompleks, sehingga cocok digunakan sebagai pengujian awal terhadap alur kompilasi dan eksekusi program. Berdasarkan output yang dihasilkan, nilai yang dicetak sesuai dengan hasil evaluasi ekspresi pada program masukan. Hal ini menunjukkan bahwa proses parsing, pembentukan intermediate code, code generation, dan interpretasi instruksi dasar telah berjalan dengan benar.

4.2. Pengujian 2

Input

```

test/milestone-4/input/2.txt

program ControlFlow;

const
    Limit == 3;
    Threshold == 5;

var
    i, total: integer;

begin
    total := 0;
    i := 1;

    while i <= Limit do
        begin
            total := total + i;
            i := i + 1;
        end;

        if total > Threshold then
            writeln('large ', total)
        else
            writeln('small ', total);

            if total mod 2 <> 0 then
                writeln('odd ', total)
            else
                writeln('even ', total);
        end.
end.

```

Output

```

test/milestone-4/output/2.txt

Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 INT 0 5
1 LIT 0 0
2 STO 0 4
3 LIT 0 1
4 STO 0 3
5 LOD 0 3

```

```
6 LIT 0 3
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 LOD 0 4
19 LIT 0 5
20 OPR 0 11
21 JPC 0 27
22 LIT 0 "large "
23 OPR 0 13
24 LOD 0 4
25 OPR 0 14
26 JMP 0 31
27 LIT 0 "small "
28 OPR 0 13
29 LOD 0 4
30 OPR 0 14
31 LOD 0 4
32 LIT 0 2
33 OPR 0 6
34 LIT 0 0
35 OPR 0 8
36 JPC 0 42
37 LIT 0 "odd "
38 OPR 0 13
39 LOD 0 4
40 OPR 0 14
41 JMP 0 46
42 LIT 0 "even "
43 OPR 0 13
44 LOD 0 4
45 OPR 0 14
46 RET 0 0
```

Codegen diagnostics:

No code generation errors.

Program Output:

large 6
even 6

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 2.txt

Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 INT 0 5
1 LIT 0 0
2 STO 0 4
3 LIT 0 1
4 STO 0 3
5 LOD 0 3
6 LIT 0 3
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 LOD 0 4
19 LIT 0 5
20 OPR 0 11
21 JPC 0 27
22 LIT 0 "large "
23 OPR 0 13
24 LOD 0 4
25 OPR 0 14
26 JMP 0 31
27 LIT 0 "small "
28 OPR 0 13
29 LOD 0 4
30 OPR 0 14
31 LOD 0 4
32 LIT 0 2
33 OPR 0 6
34 LIT 0 0
35 OPR 0 8
36 JPC 0 42
37 LIT 0 "odd "
38 OPR 0 13
39 LOD 0 4
40 OPR 0 14
41 JMP 0 46
42 LIT 0 "even "
43 OPR 0 13
44 LOD 0 4
45 OPR 0 14
46 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
large 6
even 6

Runtime diagnostics:
No runtime errors.

Status: SUCCESS
```

Penjelasan

Pengujian ini digunakan untuk memeriksa kemampuan program dalam menangani deklarasi konstanta, perulangan `while`, percabangan `if-else`, operator relasional, serta operasi modulo. Program menghitung nilai total melalui proses iteratif, kemudian mengevaluasi apakah hasilnya memenuhi kondisi tertentu dan apakah bilangan tersebut termasuk ganjil atau genap. Output yang dihasilkan menunjukkan bahwa nilai total berhasil dihitung secara akurat dan percabangan dieksekusi sesuai kondisi yang benar. Dengan demikian, test case ini membuktikan bahwa kombinasi antara konstanta, ekspresi aritmetika, ekspresi boolean, dan struktur kontrol telah diimplementasikan dengan baik.

4.3. Pengujian 3

Input
<pre>test/milestone-4/input/5.txt program ShortCircuit; var ok, skipped: boolean; begin ok := true or (1 div 0 > 0); skipped := false and (1 div 0 > 0); if ok and not skipped then writeln('short') else writeln('bad'); end.</pre>
Output
<pre>test/milestone-4/output/5.txt Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 INT 0 5 1 LIT 0 true 2 JPC 0 5 3 LIT 0 true 4 JMP 0 14 5 LIT 0 1 6 LIT 0 0</pre>

```
7 OPR 0 5
8 LIT 0 0
9 OPR 0 11
10 JPC 0 13
11 LIT 0 true
12 JMP 0 14
13 LIT 0 false
14 STO 0 3
15 LIT 0 false
16 JPC 0 25
17 LIT 0 1
18 LIT 0 0
19 OPR 0 5
20 LIT 0 0
21 OPR 0 11
22 JPC 0 25
23 LIT 0 true
24 JMP 0 26
25 LIT 0 false
26 STO 0 4
27 LOD 0 3
28 JPC 0 35
29 LOD 0 4
30 LIT 0 false
31 OPR 0 7
32 JPC 0 35
33 LIT 0 true
34 JMP 0 36
35 LIT 0 false
36 JPC 0 40
37 LIT 0 "short"
38 OPR 0 14
39 JMP 0 42
40 LIT 0 "bad"
41 OPR 0 14
42 RET 0 0
```

Codegen diagnostics:

No code generation errors.

Program Output:

short

Runtime diagnostics:

No runtime errors.

Status: SUCCESS

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 5.txt
Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 INT 0 5
1 LIT 0 true
2 JPC 0 5
3 LIT 0 true
4 JMP 0 14
5 LIT 0 1
6 LIT 0 0
7 OPR 0 5
8 LIT 0 0
9 OPR 0 11
10 JPC 0 13
11 LIT 0 true
12 JMP 0 14
13 LIT 0 false
14 STO 0 3
15 LIT 0 false
16 JPC 0 25
17 LIT 0 1
18 LIT 0 0
19 OPR 0 5
20 LIT 0 0
21 OPR 0 11
22 JPC 0 25
23 LIT 0 true
24 JMP 0 26
25 LIT 0 false
26 STO 0 4
27 LOD 0 3
28 JPC 0 35
29 LOD 0 4
30 LIT 0 false
31 OPR 0 7
32 JPC 0 35
33 LIT 0 true
34 JMP 0 36
35 LIT 0 false
36 JPC 0 40
37 LIT 0 "short"
38 OPR 0 14
39 JMP 0 42
40 LIT 0 "bad"
41 OPR 0 14
42 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
short

Runtime diagnostics:
No runtime errors.

Status: SUCCESS
```

Penjelasan

Pengujian ini bertujuan untuk memastikan bahwa ekspresi boolean dengan operator `and`, `or`, dan `not` dievaluasi menggunakan mekanisme short-circuit. Pada input, terdapat ekspresi yang secara eksplisit mengandung operasi pembagian dengan nol, tetapi operasi tersebut seharusnya tidak dievaluasi karena hasil ekspresi boolean sudah dapat ditentukan dari operand sebelumnya. Output program menunjukkan bahwa program tetap berjalan normal dan menghasilkan keluaran yang sesuai tanpa memunculkan runtime error pembagian dengan nol. Hal ini menandakan bahwa implementasi evaluasi ekspresi boolean sudah benar, khususnya dalam menghindari evaluasi ekspresi yang tidak diperlukan.

4.4. Pengujian 4

Input
<pre>test/milestone-4/input/8.txt program FunctionByReferenceParameter; var value: integer; result: integer; function AddAndReturn(var target: integer; amount: integer): integer; begin target := target + amount; AddAndReturn := target; end; begin value := 4; result := AddAndReturn(value, 3); writeln('value ', value); writeln('result ', result); end.</pre>
Output
<pre>test/milestone-4/output/8.txt Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 JMP 0 9 1 INT 0 6 2 LOD 0 3 ; indirect 3 LOD 0 4 4 OPR 0 2 5 STO 0 3 ; indirect 6 LOD 0 3 ; indirect 7 STO 0 5 8 RET 0 0 9 INT 0 5</pre>

```
10 LIT 0 4
11 STO 0 3
12 LDA 0 3
13 LIT 0 3
14 CAL 0 1
15 STO 0 4
16 LIT 0 "value "
17 OPR 0 13
18 LOD 0 3
19 OPR 0 14
20 LIT 0 "result "
21 OPR 0 13
22 LOD 0 4
23 OPR 0 14
24 RET 0 0
```

Codegen diagnostics:
No code generation errors.

Program Output:
value 7
result 7

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

Screenshot

```

Enter input file name from test/milestone-4/input (0 to go back, q to exit): 8.txt

Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 JMP 0 9
1 INT 0 6
2 LOD 0 3 ; indirect
3 LOD 0 4
4 OPR 0 2
5 STO 0 3 ; indirect
6 LOD 0 3 ; indirect
7 STO 0 5
8 RET 0 0
9 INT 0 5
10 LIT 0 4
11 STO 0 3
12 LDA 0 3
13 LIT 0 3
14 CAL 0 1
15 STO 0 4
16 LIT 0 "value "
17 OPR 0 13
18 LOD 0 3
19 OPR 0 14
20 LIT 0 "result "
21 OPR 0 13
22 LOD 0 4
23 OPR 0 14
24 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
value 7
result 7

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

```

Penjelasan

Pengujian ini digunakan untuk memverifikasi pemanggilan fungsi atau prosedur yang melibatkan parameter by-reference. Program masukan mengirimkan variabel sebagai argumen, kemudian nilai variabel tersebut dimodifikasi di dalam subprogram. Output yang dihasilkan memperlihatkan bahwa perubahan nilai di dalam subprogram ikut tercermin pada variabel asal di scope pemanggil. Dengan demikian, test case ini menunjukkan bahwa mekanisme activation record, passing parameter by-reference, serta pengelolaan alamat memori antar-scope telah berjalan sesuai spesifikasi.

4.5. Pengujian 5

Input

```
test/milestone-4/input/12.txt
```

```
program OrdinalArrayIndex;
```

```
type
```

```
    LetterRange == 'a'..'c';
```

```
    LetterMap == array[LetterRange] of integer;
```

```
    Color == (Red, Green, Blue);
```

```
var
```

```
    table: LetterMap;
```

```
    key: char;
```

```
    shade: Color;
```

```
begin
```

```
    key := 'b';
```

```
    shade := Green;
```

```
    table['a'] := 4;
```

```
    table[key] := table['a'] + 1;
```

```
    writeln('ordinal ', table['b']);
```

```
    if shade == Green then
```

```
        writeln('enum ok');
```

```
end.
```

Output

```
test/milestone-4/output/12.txt
```

```
Semantic diagnostics:
```

```
No semantic errors or warnings.
```

```
Intermediate Code:
```

```
0 INT 0 8
```

```
1 LIT 0 'b'
```

```
2 STO 0 6
```

```
3 LIT 0 1
```

```
4 STO 0 7
```

```
5 LIT 0 4
```

```
6 LDA 0 3
```

```
7 LIT 0 'a'
```

```
8 BND 97 99
```

```
9 LIT 0 97
```

```
10 OPR 0 3
```

```
11 OPR 0 2
```

```
12 STI 0 0
```

```
13 LDA 0 3
14 LIT 0 'a'
15 BND 97 99
16 LIT 0 97
17 OPR 0 3
18 OPR 0 2
19 LDI 0 0
20 LIT 0 1
21 OPR 0 2
22 LDA 0 3
23 LOD 0 6
24 BND 97 99
25 LIT 0 97
26 OPR 0 3
27 OPR 0 2
28 STI 0 0
29 LIT 0 "ordinal "
30 OPR 0 13
31 LDA 0 3
32 LIT 0 'b'
33 BND 97 99
34 LIT 0 97
35 OPR 0 3
36 OPR 0 2
37 LDI 0 0
38 OPR 0 14
39 LOD 0 7
40 LIT 0 1
41 OPR 0 7
42 JPC 0 45
43 LIT 0 "enum ok"
44 OPR 0 14
45 RET 0 0
```

Codegen diagnostics:

No code generation errors.

Program Output:

ordinal 5

enum ok

Runtime diagnostics:

No runtime errors.

Status: SUCCESS

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 12.txt

Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 INT 0 8
1 LIT 0 'b'
2 STO 0 6
3 LIT 0 1
4 STO 0 7
5 LIT 0 4
6 LDA 0 3
7 LIT 0 'a'
8 BND 97 99
9 LIT 0 97
10 OPR 0 3
11 OPR 0 2
12 STI 0 0
13 LDA 0 3
14 LIT 0 'a'
15 BND 97 99
16 LIT 0 97
17 OPR 0 3
18 OPR 0 2
19 LDI 0 0
20 LIT 0 1
21 OPR 0 2
22 LDA 0 3
23 LOO 0 6
24 BND 97 99
25 LIT 0 97
26 OPR 0 3
27 OPR 0 2
28 STI 0 0
29 LIT 0 "ordinal "
30 OPR 0 13
31 LDA 0 3
32 LIT 0 'b'
33 BND 97 99
34 LIT 0 97
35 OPR 0 3
36 OPR 0 2
37 LDI 0 0
38 OPR 0 14
39 LOO 0 7
40 LIT 0 1
41 OPR 0 7
42 JPC 0 45
43 LIT 0 "enum ok"
44 OPR 0 14
45 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
ordinal 5
enum ok

Runtime diagnostics:
No runtime errors.

Status: SUCCESS
```

Penjelasan

Pengujian ini berfokus pada penggunaan tipe ordinal sebagai indeks array, termasuk penggunaan karakter dan enumerasi. Program mendeklarasikan range karakter sebagai indeks array, mengisi salah satu elemennya, kemudian mengakses elemen tersebut menggunakan variabel bertipe karakter. Selain itu, program juga memeriksa nilai enumerasi melalui percabangan. Output yang sesuai menunjukkan bahwa compiler mampu memetakan tipe ordinal ke indeks internal array dengan benar dan mampu membandingkan nilai enumerasi secara tepat. Hal ini membuktikan bahwa dukungan terhadap tipe data bentukan ordinal tidak hanya berjalan pada deklarasi, tetapi juga pada eksekusi program.

4.6. Pengujian 6

Input
<pre>test/milestone-4/input/16.txt program AggregateByReferenceCopy; type Point == record x: integer; y: integer; end; var source: Point; target: Point; procedure CopyPoint(var destination: Point; var origin: Point); begin destination := origin; end; begin source.x := 2; source.y := 9; CopyPoint(target, source); source.y := 100; writeln('ref aggregate ', target.x + target.y); end.</pre>
Output
<pre>test/milestone-4/output/16.txt Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 JMP 0 6 1 INT 0 5 2 LOD 0 4 3 LOD 0 3 4 CPY 0 2</pre>


```
5 RET 0 0
6 INT 0 7
7 LIT 0 2
8 LDA 0 3
9 STI 0 0
10 LIT 0 9
11 LDA 0 3
12 LIT 0 1
13 OPR 0 2
14 STI 0 0
15 LDA 0 5
16 LDA 0 3
17 CAL 0 1
18 LIT 0 100
19 LDA 0 3
20 LIT 0 1
21 OPR 0 2
22 STI 0 0
23 LIT 0 "ref aggregate "
24 OPR 0 13
25 LDA 0 5
26 LDI 0 0
27 LDA 0 5
28 LIT 0 1
29 OPR 0 2
30 LDI 0 0
31 OPR 0 2
32 OPR 0 14
33 RET 0 0
```

Codegen diagnostics:
No code generation errors.

Program Output:
ref aggregate 11

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 16.txt
```

```
Intermediate code and interpreter result:
```

```
Semantic diagnostics:
```

```
No semantic errors or warnings.
```

```
Intermediate Code:
```

```
0 JMP 0 6
1 INT 0 5
2 LOD 0 4
3 LOD 0 3
4 CPY 0 2
5 RET 0 0
6 INT 0 7
7 LIT 0 2
8 LDA 0 3
9 STI 0 0
10 LIT 0 9
11 LDA 0 3
12 LIT 0 1
13 OPR 0 2
14 STI 0 0
15 LDA 0 5
16 LDA 0 3
17 CAL 0 1
18 LIT 0 100
19 LDA 0 3
20 LIT 0 1
21 OPR 0 2
22 STI 0 0
23 LIT 0 "ref aggregate "
24 OPR 0 13
25 LDA 0 5
26 LDI 0 0
27 LDA 0 5
28 LIT 0 1
29 OPR 0 2
30 LDI 0 0
31 OPR 0 2
32 OPR 0 14
33 RET 0 0
```

```
Codegen diagnostics:
```

```
No code generation errors.
```

```
Program Output:
```

```
ref aggregate 11
```

```
Runtime diagnostics:
```

```
No runtime errors.
```

```
Status: SUCCESS
```

Penjelasan

Pengujian ini bertujuan untuk memastikan bahwa struktur data agregat, seperti array atau record, dapat digunakan dalam operasi penyalinan dan pengiriman parameter by-reference. Program pada test case ini memanipulasi data terstruktur dan mengecek apakah perubahan terhadap data tersebut tetap konsisten setelah melewati proses assignment maupun pemanggilan subprogram. Output yang dihasilkan sesuai dengan nilai yang diharapkan, sehingga dapat disimpulkan bahwa layout memori untuk data agregat telah dikelola dengan benar. Test case ini penting karena kesalahan pada implementasi aggregate value biasanya tidak terlihat pada tipe primitif, tetapi dapat menyebabkan data tertimpa atau alamat memori tidak sesuai pada struktur yang lebih kompleks.

4.7. Pengujian 7

Input
<pre>test/milestone-4/input/17.txt program BasicInput; var n: integer; amount: real; flag: boolean; letter: char; word: string; begin readln; read(n, amount); readln(flag, letter, word); writeln('n ', n + 1); writeln('amount ', amount); writeln('flag ', flag); writeln('letter ', letter); writeln('word ', word); end.</pre>
Output
<pre>Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 INT 0 8 1 INL 0 1 2 LDA 0 3 3 INP 0 1 4 LDA 0 4 5 INP 0 2 6 LDA 0 5 7 INP 0 4 8 LDA 0 6 9 INP 0 3 10 LDA 0 7 11 INP 0 5</pre>

```
12 INL 0 0
13 LIT 0 "n "
14 OPR 0 13
15 LOD 0 3
16 LIT 0 1
17 OPR 0 2
18 OPR 0 14
19 LIT 0 "amount "
20 OPR 0 13
21 LOD 0 4
22 OPR 0 14
23 LIT 0 "flag "
24 OPR 0 13
25 LOD 0 5
26 OPR 0 14
27 LIT 0 "letter "
28 OPR 0 13
29 LOD 0 6
30 OPR 0 14
31 LIT 0 "word "
32 OPR 0 13
33 LOD 0 7
34 OPR 0 14
35 RET 0 0
```

Codegen diagnostics:
No code generation errors.

Program Output:
n 5
amount 2.5
flag true
letter Z
word hello

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

Screenshot

```

Enter input file name from test/milestone-4/input (0 to go back, q to exit): 17.txt
skip this line
4 2.5 true Z hello

Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 INT 0 8
1 INL 0 1
2 LDA 0 3
3 INP 0 1
4 LDA 0 4
5 INP 0 2
6 LDA 0 5
7 INP 0 4
8 LDA 0 6
9 INP 0 3
10 LDA 0 7
11 INP 0 5
12 INL 0 0
13 LIT 0 "n "
14 OPR 0 13
15 LOD 0 3
16 LIT 0 1
17 OPR 0 2
18 OPR 0 14
19 LIT 0 "amount "
20 OPR 0 13
21 LOD 0 4
22 OPR 0 14
23 LIT 0 "flag "
24 OPR 0 13
25 LOD 0 5
26 OPR 0 14
27 LIT 0 "letter "
28 OPR 0 13
29 LOD 0 6
30 OPR 0 14
31 LIT 0 "word "
32 OPR 0 13
33 LOD 0 7
34 OPR 0 14
35 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
n 5
amount 2.5
flag true
letter Z
word hello

Runtime diagnostics:
No runtime errors.

Status: SUCCESS

```

Penjelasan

Pengujian ini dilakukan untuk memastikan bahwa program dapat membaca input dari pengguna menggunakan `read` dan `readln` untuk berbagai tipe data. Input runtime yang diberikan berisi nilai integer, real, boolean, character, dan string yang seluruhnya valid terhadap tipe variabel tujuan. Output program menunjukkan bahwa setiap nilai berhasil dibaca dan ditampilkan kembali sesuai bentuk serta urutan yang diharapkan. Dengan demikian, test case ini membuktikan bahwa mekanisme pembacaan input, konversi tipe, serta penyimpanan nilai hasil input ke memori telah berjalan dengan benar untuk kasus valid.

4.8. Pengujian 8

Input
<pre>test/milestone-4/input/19.txt program InvalidRuntimeInput; var n: integer; begin readln(n); writeln('unreachable ', n); end.</pre>
Output
<pre>test/milestone-4/output/19.txt Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 INT 0 4 1 LDA 0 3 2 INP 0 1 3 INL 0 0 4 LIT 0 "unreachable " 5 OPR 0 13 6 LOD 0 3 7 OPR 0 14 8 RET 0 0 Codegen diagnostics: No code generation errors. Program Output: <no output> Runtime diagnostics: Runtime error at instruction 2: invalid input token 'abc' Status: FAILED</pre>

Screenshot
<pre> Enter input file name from test/milestone-4/input (0 to go back, q to exit): 19.txt abc Intermediate code and interpreter result: Semantic diagnostics: No semantic errors or warnings. Intermediate Code: 0 INT 0 4 1 LDA 0 3 2 INP 0 1 3 INL 0 0 4 LIT 0 "unreachable " 5 OPR 0 13 6 LOD 0 3 7 OPR 0 14 8 RET 0 0 Codegen diagnostics: No code generation errors. Program Output: <no output> Runtime diagnostics: Runtime error at instruction 2: invalid input token 'abc' Status: FAILED </pre>
Penjelasan
Pengujian ini bertujuan untuk memeriksa mekanisme error handling ketika input runtime tidak sesuai dengan tipe data yang diminta oleh program. Pada test case ini, program mencoba membaca nilai bertipe integer, tetapi input yang diberikan berupa token non-numerik. Output yang muncul berupa pesan runtime error yang menjelaskan bahwa token input tidak valid. Hasil tersebut menunjukkan bahwa interpreter tidak memaksakan konversi data yang salah dan mampu menghentikan eksekusi secara aman ketika terjadi ketidaksesuaian tipe input.

4.9. Pengujian 9

Input
<pre> test/milestone-4/input/22.txt program InvalidStructuredValueContexts; type Point == record x: integer; y: integer; end; </pre>

```

var
    p: Point;

function MakePoint: Point;
begin
    MakePoint.x := 1;
    MakePoint.y := 2;
end;

begin
    writeln(p);
    readln(p);
end.

```

Output

test/milestone-4/output/22.txt

Semantic diagnostics:

Error at line 12, column 21: function return type cannot be an array or record

Error at line 19, column 13: argument 1 of 'writeln' cannot be a structured value

Error at line 20, column 12: argument 1 of 'readln' cannot be a structured target

Intermediate Code:

<not generated because semantic analysis failed>

Codegen diagnostics:

<not run because semantic analysis failed>

Program Output:

<not executed>

Runtime diagnostics:

<not run because semantic analysis failed>

Status: FAILED

Screenshot

<pre> Enter input file name from test/milestone-4/input (0 to go back, q to exit): 22.txt Intermediate code and interpreter result: Semantic diagnostics: Error at line 12, column 21: function return type cannot be an array or record Error at line 19, column 13: argument 1 of 'writeln' cannot be a structured value Error at line 20, column 12: argument 1 of 'readln' cannot be a structured target Intermediate Code: <not generated because semantic analysis failed> Codegen diagnostics: <not run because semantic analysis failed> Program Output: <not executed> Runtime diagnostics: <not run because semantic analysis failed> Status: FAILED </pre>
Penjelasan
Pengujian ini digunakan untuk memastikan bahwa semantic analyzer dapat menolak penggunaan nilai terstruktur pada konteks yang tidak diperbolehkan. Program pada input mengandung operasi atau penggunaan structured value yang tidak sesuai dengan aturan bahasa, sehingga seharusnya terdeteksi sebelum tahap eksekusi. Output yang dihasilkan berupa pesan error semantik, bukan hasil eksekusi program. Hal ini menunjukkan bahwa pemeriksaan semantik telah bekerja dengan benar dalam mencegah program invalid masuk ke tahap code generation atau interpreter.

4.10. Pengujian 10

Input
<pre> test/milestone-4/input/24.txt program RuntimeStackOverflow; procedure Loop; begin Loop; end; begin Loop; end. </pre>
Output
<pre> test/milestone-4/input/24.txt Semantic diagnostics: No semantic errors or warnings. </pre>

Intermediate Code:

```
0 JMP 0 4
1 INT 0 3
2 CAL 1 1
3 RET 0 0
4 INT 0 3
5 CAL 0 1
6 RET 0 0
```

Codegen diagnostics:

No code generation errors.

Program Output:

<no output>

Runtime diagnostics:

Runtime error at instruction 2: CAL: stack overflow: call depth exceeds limit 1000

Status: FAILED

Screenshot

```
Enter input file name from test/milestone-4/input (0 to go back, q to exit): 24
Intermediate code and interpreter result:
Semantic diagnostics:
No semantic errors or warnings.

Intermediate Code:
0 JMP 0 4
1 INT 0 3
2 CAL 1 1
3 RET 0 0
4 INT 0 3
5 CAL 0 1
6 RET 0 0

Codegen diagnostics:
No code generation errors.

Program Output:
<no output>

Runtime diagnostics:
Runtime error at instruction 2: CAL: stack overflow: call depth exceeds limit 1000

Status: FAILED
```

Penjelasan

Pengujian ini bertujuan untuk memeriksa keamanan interpreter terhadap pemanggilan subprogram rekursif yang tidak

memiliki kondisi berhenti. Program pada input melakukan pemanggilan prosedur secara terus-menerus sehingga kedalaman call stack akan bertambah tanpa batas apabila tidak ada mekanisme pembatasan. Output yang dihasilkan menunjukkan bahwa interpreter mendeteksi kondisi stack overflow atau batas kedalaman pemanggilan dan menghentikan eksekusi dengan pesan error yang sesuai. Dengan demikian, test case ini membuktikan bahwa program tidak hanya menjalankan kasus normal, tetapi juga memiliki perlindungan terhadap skenario runtime yang berpotensi menyebabkan crash atau penggunaan memori berlebihan.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Pada milestone 4 ini, program Arion Compiler telah berhasil dikembangkan dari tahap semantic analysis menuju intermediate code generation dan interpreter. Program tidak hanya mampu memvalidasi source code secara leksikal, sintaksis, dan semantik, tetapi juga mampu mengubah decorated AST menjadi intermediate code yang dapat dieksekusi oleh stack machine interpreter.

Intermediate code generator yang dibuat telah mampu menerjemahkan berbagai struktur program Arion, seperti assignment, ekspresi aritmetika, ekspresi boolean, percabangan, perulangan, procedure, function, parameter by-value dan by-reference, array, record, serta operasi input/output. Instruksi yang dihasilkan kemudian dijalankan oleh interpreter dengan mengelola instruction pointer, operand stack, memory, stack frame, static link, dynamic link, return address, dan nilai return function.

Berdasarkan hasil pengujian, program dapat menghasilkan intermediate code dan output akhir yang sesuai untuk input valid. Program juga mampu menangani beberapa kasus kesalahan, seperti input runtime yang tidak valid, penggunaan structured value pada konteks yang salah, serta stack overflow akibat rekursi tanpa kondisi berhenti. Dengan demikian, program telah memenuhi tujuan milestone 4, yaitu menghasilkan intermediate code dan mengeksekusi program Arion hingga menghasilkan output akhir.

5.2. Saran

Pelaksanaan Milestone 4 Tugas Besar Teori Bahasa Formal dan Automata di Semester II (Genap) Tahun 2025/2026 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan.

Untuk pengembangan selanjutnya, intermediate code generator dan interpreter dapat dilengkapi dengan optimisasi instruksi agar intermediate code yang dihasilkan menjadi lebih ringkas dan efisien. Selain itu, error handling pada runtime juga dapat

dikembangkan lebih lanjut agar pesan kesalahan menjadi lebih detail dan mudah dipahami oleh pengguna.

Pengujian juga dapat diperluas dengan lebih banyak variasi kasus, terutama untuk kombinasi nested procedure/function, array dan record bertingkat, input yang lebih kompleks, serta skenario runtime error lainnya. Dengan pengujian yang lebih beragam, keandalan interpreter dan stack machine dapat dipastikan dengan lebih baik.

BAB VI

LAMPIRAN

6.1. Lampiran Link Repository Github.

Link : <https://github.com/ucupsss/SMG-Tubes-IF2224-2026>

6.2. Pembagian Tugas

NIM	Nama	Tugas	Kontribusi
13524014	Yusuf Faishal Listyardi	Integrasi Milestone 4, Codegen Declaration, dan Laporan	25%
13524046	Farrell Limjaya	Codegen Core, Codegen Statement, dan Laporan	25%
13524066	Nathanael Gunawan	Interpreter dan Laporan	25%
13524070	A. Fawwaz Azam Wicaksono	Intermediate, Codegen Expression, dan Laporan	25%

REFERENSI

- <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0264326#pone.0264326.ref001>
- <https://opensa.cs.vt.edu/ODSA/Books/PL/html/Grammars1.html>
- <https://www.ibm.com/id-id/think/topics/compiler>
-  Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebo...
- <https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>